Expert Systems With Applications 313 (2026) 131575

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

QER-LPD3QN: A quantum-Inspired Sequence-Aware deep reinforcement learning algorithm for path planning

He Guo [a], Zhengyi Chai [b,*], Yalun Li [c]

[a] School of Control Science and Engineering, Tiangong University, No. 399, Binshui West Road, 300387, Tianjin, China
[b] School of Computer Science and Technology, Tiangong University, No. 399, Binshui West Road, 300387, Tianjin, China
[c] School of Electronic and Information Engineering, Tiangong University, No. 399, Binshui West Road, 300387, Tianjin, China

ARTICLE INFO

Keywords:
Path planning
Autonomous navigation
Reinforcement learning
Dynamic environments
Quantum-Inspired replay

ABSTRACT

Real-time, safe, and smooth path planning in highly dynamic scenes remains challenging. Temporal evolution is under-modeled by classical planners, while standard deep reinforcement learning suffers from temporal inconsistency in experience replay and Q-value overestimation. This paper proposes QER-LPD3QN, a novel framework that combines Quantum-Inspired Sequence Experience Replay (QER) with an LSTM-based Dueling Double DQN. In QER, each experience sequence is encoded as a qubit-like state with probability amplitudes defining sampling priorities. Two rotation operators are introduced: "prepare" amplifies samples with high TD error, while "depreciate" down-weights over-replayed samples, thereby preserving temporal coherence and enabling automatic priority adaptation. The LSTM backbone captures long-term dependencies, and the Dueling-Double architecture stabilizes value estimation. Extensive evaluations in static benchmarks and highly dynamic benchmarks (e.g., "Rotating Broom", "Dynamic Traffic") demonstrate that our method maintains real-time performance with average inference latency below 4 ms. Unlike classical planners and standard on-policy algorithms (e.g., PPO) that fail to reliably navigate dynamic obstacles, our approach achieves perfect safety records with zero collisions in all evaluated dynamic environments. Quantitative results show significantly smoother trajectories compared to IDDQN and other reinforcement learning baselines. Ablation studies confirm the contributions of individual components: LSTM enhances convergence stability, QER provides better sample efficiency than Prioritized Experience Replay (PER), and the Dueling-Double design effectively reduces estimation bias. The proposed framework is controller-agnostic and particularly suited to non-stationary environments, enabling robust autonomous navigation.

1. Introduction

With recent advances in artificial intelligence, embodied intelligence has garnered significant attention. Agents that interact with the physical world require robust navigation capabilities; examples include service robots, autonomous vehicles, and quadruped robots. Fundamental to this interaction is path planning, where the objective is to generate safe and efficient routes from a starting point to a destination while avoiding obstacles.

Prior research typically categorizes path planning into global and local planning. Global planning has been extensively investigated, often relying on the assumption of pre-known static obstacle maps. In contrast, local planning operates under stricter constraints, including limited sensing ranges, sensor noise, and the restricted computational power of edge devices. Consequently, navigating complex environments populated with dynamic obstacles remains a challenging task. While rule-based and optimization-based local planners have been widely applied, guaranteeing real-time performance and robustness in highly dynamic, partially observable scenes is difficult due to the computational complexity of recalculating trajectories on the fly Carvalho et al. (2025).

To address these limitations, reinforcement learning (RL) has been increasingly adopted. By formulating path planning as a Markov decision process (MDP), deep reinforcement learning (DRL) integrates deep neural networks to extract high-dimensional environmental features and select actions directly. This end-to-end approach has demonstrated the ability to learn complex policies that adapt better to dynamic environments compared to classical separation of perception and planning. However, DRL applications face persistent challenges, including slow training convergence, low sample efficiency, and instability. Although Prioritized Experience Replay (PER) is commonly used to improve sample utilization by replaying transitions with high temporal-difference (TD) errors, it introduces new issues in sequential tasks.

* Corresponding author.
E-mail addresses: 936515285@qq.com (H. Guo), chaizhengyi@tiangong.edu.cn (Z. Chai), liyalun@tiangong.edu.cn (Y. Li).

https://doi.org/10.1016/j.eswa.2026.131575
Received 24 September 2025; Received in revised form 16 January 2026; Accepted 5 February 2026
Available online 7 February 2026

Path planning is inherently a sequential decision-making problem where temporal correlation matters. Standard PER treats experiences as independent and identically distributed (i.i.d.) samples, disrupting the temporal dependencies required for tasks like motion prediction and smooth obstacle avoidance Li et al. (2025). Furthermore, conventional PER lacks a dynamic mechanism to adjust the value of experiences over time; misleading transitions may be oversampled, inducing training instability and performance oscillations.

In parallel, the intersection of quantum computing and artificial intelligence has provided new theoretical perspectives Hakemi et al. (2024). Specifically, probability-amplitude encoding in qubits offers a mechanism to enhance population diversity and global search efficiency. In the context of reinforcement learning, quantum parallelism and interference concepts have been leveraged to balance exploration and exploitation, thereby accelerating convergence. Recently, Wei et al. proposed DRL-QER, combining deep reinforcement learning with quantum-inspired experience replay Wei et al. (2024). In their approach, experiences are mapped to quantum states, and their replay probabilities are iteratively adjusted via "prepare" and "depreciate" operators. However, standard DRL-QER operates on single transitions. This reduction of a sequential problem to discrete sampling breaks temporal correlations, making current quantum-inspired methods difficult to apply effectively to long-horizon path-planning tasks.

To address the issues of low sample efficiency, broken temporal dependencies, and unstable convergence in dynamic environments, this paper proposes a unified framework. We extend the quantum-inspired mechanism to the sequence level to preserve trajectory continuity. The main contributions are summarized as follows:

1. **QER-LPD3QN Framework.** We introduce a unified navigation framework by integrating Quantum-Inspired Sequence Experience Replay (QER) with an LSTM-based dueling double DQN. The LSTM module enables temporal modeling, while the dueling–double architecture stabilizes value estimation. This combination reduces the trajectory smoothness cost by approximately **87.5%** relative to non-recurrent baselines (IDDQN), based on averaged results over multiple random seeds.
2. **Sequence-Level Quantum-State Prioritization.** We design a sequence-level *prepare–depreciate* rotation mechanism to adaptively regulate replay priorities. Unlike PER-based methods, this scheme preserves temporal continuity and avoids oversampling short-range fragments. The resulting replay distribution improves path-planning efficiency by **5.4%** compared with a sequence-aware PER baseline (PER_LSTM).
3. **Systematic Evaluation and Real-Time Feasibility.** We conduct extensive tests in dense static maps and highly dynamic scenarios, including the “Rotating Broom” and multi-flow intersections. QER-LPD3QN maintains an average inference latency below **4 ms** and achieves a **100%** success rate under the evaluated settings, outperforming PPO by **20%** in dynamic-scene success rate while exhibiting zero collisions across all runs.

The remainder of this paper is organized as follows. Section II reviews related work on path planning, DRL-based planners, and experience replay strategies. Section III formalizes the path-planning MDP and safety constraints. Section IV details the QER-LPD3QN framework, including the quantum-state sequence replay mechanism. Section V reports comparative experiments and ablation studies. Finally, Section VI concludes the paper and discusses future directions, such as multi-robot coordination and sim-to-real transfer.

2. Related work and our research motivation

2.1. Classical path planning algorithms

Classical graph-search and sampling-based planners provide explicit guarantees in static or well-structured environments. Optimal paths can be obtained by A* when an admissible heuristic is used Tang et al. (2021). RRT ensures probabilistic completeness in high-dimensional spaces, whereas RRT* achieves asymptotic optimality and improves path quality in complex scenes Zhang et al. (2024). For local obstacle avoidance and real-time control, Artificial Potential Fields (APF) and the Dynamic Window Approach (DWA) are widely adopted. In APF, motion is defined in potential space; in DWA, search is performed in velocity space. Fast reactions are achieved by both methods, yet local minima, oscillatory behaviours, and suboptimal solutions frequently arise under strong dynamics or locally extreme conditions Zhang et al. (2025b).

Despite their advantages, classical planners exhibit several limitations. Global graph-search methods become computationally expensive in high-dimensional or densely cluttered environments. Sampling-based planners often generate jagged or suboptimal trajectories near dense obstacles due to their stochastic nature. Reactive local methods such as APF and DWA remain vulnerable to local minima and cannot reliably handle long-horizon interactions. Moreover, partial observability and temporal dependencies are not explicitly modelled, which restricts their applicability in fast-changing, highly dynamic scenes.

2.2. Deep reinforcement learning for path planning and navigation

Deep reinforcement learning (DRL) offers distinct advantages for path planning and autonomous navigation compared to classical approaches. First, end-to-end frameworks enable a unified perception-decision-control pipeline. Raw sensory observations, such as laser scans or camera images, can be mapped directly to control commands without the need for explicit geometric reconstruction or map building. Second, DRL possesses strong representation learning capabilities. This allows policies to capture complex, non-linear relationships between environmental features and agent actions, facilitating robust adaptability to dynamic and partially observable environments where traditional model-based planners often struggle.

Specifically, widely adopted value-based methods enhance robustness through architectural innovations: Double DQN mitigates Q-value overestimation, while the Dueling architecture separates state value from action advantage to stabilize estimation. Similarly, policy-gradient methods like PPO employ clipped surrogate objectives to balance training stability with sample efficiency, leading to their wide adoption in continuous control tasks Wang et al. (2025). Furthermore, mechanisms like Prioritized Experience Replay (PER) significantly improve sample utilization by focusing learning on highly informative transitions Zhou et al. (2024), Gök (2024).

However, despite these strengths, existing DRL-based planners face limitations in sequence modeling. Standard replay mechanisms often disrupt temporal dependencies, leading to myopic decisions in rapidly changing scenes. Over-replayed transitions can introduce estimation bias, while sequence fragmentation hinders the learning of consistent trajectories. These limitations motivate the development of the proposed sequence-coherent, sample-efficient QER framework to ensure safe and smooth navigation in highly dynamic environments.

2.3. Quantum-Inspired algorithms

Quantum-inspired (QI) methods do not rely on quantum hardware. Ideas of superposition, probability amplitudes, and rotation gates are transferred to classical algorithms. QEA and QPSO replace conventional encodings or velocity updates with qubit–amplitude parameterization. Strong global search and robustness have been reported Deng et al. (2024), Dai and Zhou (2023).

The coupling of QI with reinforcement learning has deepened. In edge computing and task offloading, QI-modified RL has improved exploration and sample efficiency. Better latency–energy trade-offs have been achieved under multi-constraint optimization Wang et al. (2021), Ansere et al. (2023), Wei et al. (2024), Zhang et al. (2025a). In robotic path planning, QI has been injected into experience management and

exploration. Li et al. linked experience importance to qubit amplitudes and used Grover-style amplitude amplification to emphasize high-value samples in UAV planning Li et al. (2021). Huang et al. used random quantum walks to synthesize informative "dream" samples for world-model training. Exploration quality was improved Huang et al. (2024).

Despite these gains, systematic coupling with long-horizon sequential dependence, runtime constraints, and safety remains limited. Empirical studies also indicate conditional benefits. Performance depends on problem structure and hyperparameter matching Olvera et al. (2023). In mobile-robot local planning, replay without temporal context can induce oversampling and estimation bias. Safety margins and trajectory quality degrade in strongly dynamic environments.

2.4. Comparison and research motivation

Classical planners are mainly designed for static or slowly varying scenes. Temporal evolution is simplified or ignored. Their robustness degrades in highly dynamic environments. End-to-end DRL improves adaptability and enables mapless navigation. However, standard experience replay breaks temporal coherence. Sequences are decomposed into isolated transitions. Noisy or over-replayed samples are frequently selected. Training can become unstable, and Q-value overestimation may appear. Existing quantum-inspired methods enhance global exploration and sample efficiency. Yet, their integration with sequence modelling, strict real-time constraints, and safety-aware navigation remains limited.

These gaps motivate sequence-level experience management for DRL-based path planning. Temporal dependencies should be preserved during sampling. Sampling priorities should depend on both TD error and replay counts. Over-replay should be suppressed. Real-time operation is required. Safety margins and path smoothness should be improved under millisecond-level latency.

To address these needs, a quantum-inspired, sequence-level replay mechanism is adopted. Experience sequences are encoded as qubit-like states. Amplitudes define adaptive sampling priorities. Prepare rotations increase the amplitudes of high-error sequences. Depreciate rotations down-weight over-replayed sequences. An LSTM backbone is used to capture long-term dependencies. A dueling-double architecture is employed to reduce bias and stabilise learning. The resulting framework, QER-LPD3QN, targets safe, smooth, and real-time planning in strongly dynamic environments. In the evaluated scenarios, millisecond-level planning latency is achieved. Larger safety margins and higher trajectory quality are observed in dense obstacles, narrow intersections, and highly dynamic flows, compared with classical planners and vanilla DRL baselines. These design choices jointly define the QER-LPD3QN framework and directly implement the three contributions summarised at the end of Section 1.

3. Problem formulation

3.1. Task and workspace

The navigation task is formulated in a planar workspace $\mathcal{W} \subset \mathbb{R}^2$. A 2D top-down representation has been widely adopted in the path-planning and DRL navigation literature Zhou et al. (2024), Gök (2024), Bai et al. (2024). Although UAVs are capable of 3D motion, a planar projection is used in many existing works for the following reasons:

1. **Focus on interaction logic.** In dynamic navigation, the major difficulty is introduced by temporal reasoning and collision-risk evolution, rather than by full 3D manoeuvring. By projecting 3D obstacles onto the dominant motion plane ($\Omega \subset \mathcal{W}$), the essential interaction patterns with moving obstacles can be preserved while unnecessary degrees of freedom are avoided Gök (2024), Li et al. (2025), Fan et al. (2023). This setting enables the proposed sequence-aware replay mechanism to be evaluated under strong temporal variations.
2. **Universality and comparability.** A planar formulation is commonly used as a platform-agnostic benchmark for a broad class of mobile agents, such as ground robots, surface vehicles, and UAVs in cruise-mode navigation Zhou et al. (2024), Zhang et al. (2025b), Fan et al. (2023), Bai et al. (2024). As a result, policies can be compared fairly with established baselines under a shared geometric representation, and the proposed replay mechanism can be assessed without being coupled to a specific 3D kinematic model.

Consequently, the problem is modelled using top-down 2D maps. The UAV is represented as a rigid disc with radius r_{bot}. The inflated obstacle set is defined by the Minkowski sum $\Omega^{\oplus} = \Omega \oplus \mathbb{B}(0, r_{\text{bot}} + r_{\text{inf}})$, with boundary $\partial\Omega^{\oplus}$.

3.2. Path and metric definitions

The planned path is represented as a polyline $\mathcal{P} = \{p_i\}_{i=0}^{N} \subset \mathbb{R}^2$. To avoid discretization bias, $\mathcal{P}$ is resampled at a fixed spatial interval Δs. Let q denote a generic point on the boundary $\partial\Omega^{\oplus}$ of the inflated obstacle set.

3.2.0.1. Clearance (Safety). The pointwise clearance $d(p)$ is defined as the Euclidean distance to the nearest obstacle boundary point q, as formulated in Eq. (1):

$$d(p) = \min_{q \in \partial\Omega^{\oplus}} \|p - q\|_2. \tag{1}$$

Based on this, the minimum clearance (M_d) and average clearance (A_d) along the trajectory are defined in Eq. (2):

$$M_d = \min_{0 \le i \le N} d(p_i), \qquad A_d = \frac{1}{N+1} \sum_{i=0}^{N} d(p_i). \tag{2}$$

Consequently, the collision-free constraint is formally satisfied if and only if $M_d > 0$. Conversely, a collision event is defined by $M_d \le 0$.

3.2.0.2. Path Length (Efficiency). The total path length L is calculated as the sum of Euclidean distances between consecutive waypoints, as shown in Eq. (3):

$$L = \sum_{i=0}^{N-1} \|p_{i+1} - p_i\|_2 \quad \text{(meters)}. \tag{3}$$

3.2.0.3. Planning Time (Real-Time). Let t_i be the inference time at step i. The average planning latency T is defined in Eq. (4), representing the mean forward-pass time per step (excluding rendering):

$$T = \frac{1}{N} \sum_{i=0}^{N-1} t_i \quad \text{(milliseconds)}. \tag{4}$$

3.2.0.4. Smoothness (Quality). Let the segment direction vectors be $u_i = \frac{p_{i+1} - p_i}{\|p_{i+1} - p_i\|_2}$ for $i = 0, \ldots, N-1$. The heading change at step i is given by $\Delta\psi_i = \arccos(\text{clip}(u_{i-1}^{\top} u_i, -1, 1))$ for $i \ge 1$. The smoothness metric S_ψ (where lower values indicate smoother paths) is defined in Eq. (5):

$$S_\psi = \frac{1}{N-1} \sum_{i=1}^{N-1} \Delta\psi_i \quad \text{(radians)}. \tag{5}$$

When necessary, a curvature-based proxy S_κ is also employed, as defined in Eq. (6):

$$S_\kappa = \frac{1}{(N-1)(\Delta s)^2} \sum_{i=1}^{N-1} \|p_{i+1} - 2p_i + p_{i-1}\|_2. \tag{6}$$

Unless otherwise stated, S_ψ is adopted as the default smoothness metric in this work.

3.3. Optimization statement with metric constraints

The planning problem is posed as a constrained multi-criterion optimization. Clearance-based collision definitions and chance-constrained safety formulations are adopted following common practice in learning-based and classical path-planning studies Zhou et al. (2024), Gök (2024), Fan et al. (2023). The metrics above are incorporated as intrinsic terms in (7):

$$\min_{\pi} \quad \mathbb{E}_{\pi}\big[w_L\, L(\tau) \;+\; w_S\, S(\tau)\big] \tag{7}$$

$$\text{s.t.} \quad \Pr_{\pi}\{\text{collision before } x_g\} \le \varepsilon, \quad \text{with collision defined by}$$

$$M_d \le 0 \text{ in Eq. (2)}, \tag{8}$$

$$\mathbb{E}_{\pi}\big[M_d(\tau)\big] \;\ge\; d_{\min}, \tag{9}$$

$$\mathbb{E}_{\pi}\big[T(\tau)\big] \;\le\; \tau_{\mathrm{rt}}, \tag{10}$$

$$\Pr_{\pi}\{\text{reach } x_g \text{ within horizon } H\} \;\ge\; 1-\delta. \tag{11}$$

Here τ denotes the executed trajectory induced by policy π. Weights $w_L, w_S > 0$ define a scalarization of length and smoothness. Either $S(\cdot) = S_{\psi}(\cdot)$ or $S(\cdot) = S_{\kappa}(\cdot)$ is used, as specified. Constraints (8)-(10) bind safety and real-time performance with the metric definitions (2)-(4). The horizon H and tolerance δ define the task deadline.

3.4. Markov decision process

The path-planning problem is modeled as a Markov Decision Process (MDP), formally defined as a tuple in Eq. (12):

$$\mathcal{M} = (\mathcal{S},\, \mathcal{A},\, \mathcal{R},\, \mathcal{T},\, \gamma). \tag{12}$$

Here, $\mathcal{S}$ is the continuous state space, $\mathcal{A}$ denotes the action space, $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function, $\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ represents the state transition kernel (where $\Delta(\mathcal{S})$ denotes the set of probability distributions over $\mathcal{S}$), and $\gamma \in [0, 1)$ is the discount factor.

State Space. The state space is defined as the Cartesian product of the agent's kinematic states and environmental information, as shown in Eq. (13):

$$\mathcal{S} = \mathcal{P} \times \mathcal{V} \times \Psi \times \mathcal{G} \times \mathcal{Z}^{\mathrm{stat}} \times \mathcal{Z}^{\mathrm{dyn}}. \tag{13}$$

Consequently, a specific state $\mathbf{s}_t \in \mathcal{S}$ at time t is represented as the vector in Eq. (14):

$$\mathbf{s}_t = \big(\mathbf{p}_t,\, \mathbf{v}_t,\, \psi_t,\, \mathbf{g},\, \mathbf{z}_t^{\mathrm{stat}},\, \mathbf{z}_t^{\mathrm{dyn}}\big). \tag{14}$$

Here, $\mathbf{p}_t \in \mathcal{P} \subseteq \mathbb{R}^2$ denotes position; $\mathbf{v}_t \in \mathcal{V} \subseteq \mathbb{R}^2$ denotes velocity; $\psi_t \in \Psi \subseteq [-\pi, \pi]$ denotes heading; $\mathbf{g}$ encodes the goal coordinates; and $\mathbf{z}_t^{\mathrm{stat}}, \mathbf{z}_t^{\mathrm{dyn}}$ summarize the static and dynamic obstacle features, respectively.

Action Space. The action space $\mathcal{A}$ comprises all feasible control commands. The action at time t is denoted by $\mathbf{a}_t \in \mathcal{A}$. This continuous space allows for translational and rotational control to react to fast environmental changes.

Reward. The reward function $\mathcal{R}$ evaluates the immediate quality of an action, as defined in Eq. (15):

$$r_t = \mathcal{R}(\mathbf{s}_t, \mathbf{a}_t). \tag{15}$$

Positive rewards are assigned for progress toward the goal, while collisions and near-collisions incur penalties. Smoothness and control-effort terms are incorporated to encourage stable navigation.

Transition. The stochastic transition kernel $\mathcal{T}$ models the system dynamics and environmental uncertainties. The evolution of the system can be formally represented by the transition relation in Eq. (16):

$$\mathbf{s}_t \xrightarrow{\mathbf{a}_t} \mathbf{s}_{t+1} \quad \Longleftrightarrow \quad \mathcal{T}(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t) > 0. \tag{16}$$

To address the kinematic evolution explicitly, the transition of the agent's motion components (position, velocity, heading) follows a nonlinear dynamic model with additive noise, as formulated in Eq. (17):

$$\mathbf{s}_{t+1}^{\mathrm{kin}} = f_{\mathrm{kin}}(\mathbf{s}_t^{\mathrm{kin}}, \mathbf{a}_t) + \boldsymbol{\xi}_t, \tag{17}$$

where $\mathbf{s}_t^{\mathrm{kin}} = (\mathbf{p}_t, \mathbf{v}_t, \psi_t)$ represents the kinematic subset of the state, f_{kin} encodes the nominal vehicle dynamics, and $\boldsymbol{\xi}_t$ denotes stochastic disturbances (e.g., sensor noise or actuation error). This formulation resolves the dimensionality of the full state space $\mathcal{S}$ by focusing the stochastic differential equation on the agent's motion, avoiding the contradiction of applying kinematics to environmental features.

Objective. The action-value function is defined in Eq. (18):

$$Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k\, r_{t+k} \,\middle|\, \mathbf{s}_t, \mathbf{a}_t\right]. \tag{18}$$

The goal is to find an optimal policy π^* that maximizes the expected discounted return:

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right], \tag{19}$$

subject to the collision-free constraints inherent in the state transitions defined by $\mathcal{T}$.

4. The proposed algorithm

Section 4 presents the QER-LPD3QN framework in a structured sequence that is aligned with the computational pipeline depicted in Fig. 1. This presentation order is determined by the inherent technical dependencies among the core components. The formulation of the discrete action space in Section 4.1 serves as a foundational prerequisite, as the dimensionality and semantics of the action set directly determine the output-layer configuration of the decision network. Building upon this foundation, Section 4.2 elaborates on the LSTM-based dueling double DQN architecture, which maps encoded observations to precise action-value estimates within the predefined action space.

The network's output, specifically the temporal-difference (TD) error, subsequently becomes the primary driver for the experience replay mechanism. Consequently, Section 4.3 introduces the quantum-inspired sequence experience replay (QER) buffer, detailing how it uses the TD error generated by the network in Section 4.2 to dynamically prioritize and sample experience sequences for efficient training. This modular presentation explicitly links architectural components to their functional roles, ensuring a logical progression from action-space specification to network design and, finally, to the implementation of the specialized training mechanism.

The pipeline operates in two distinct phases: during inference (green path), observations are sequentially encoded by the network to generate actions via greedy or ϵ-greedy policies; during training (yellow path), the QER buffer dynamically updates network parameters and replay samples based on termination conditions (collision, timeout, or horizon).

4.1. Action space design for path planning

Classical planners often use a small discrete action set. Forward and turn commands are typical. This design is simple but insufficient in complex dynamic environments. Finer control is required. Therefore, QER-LPD3QN adopts a discretized version of a continuous action space.

Design considerations.

- *Discreteness vs. continuity.* Two action dimensions are defined. Step length controls forward motion per update, and turn angle controls heading change. Each dimension is discretized into bins to balance precision and complexity.
- *Smoothness.* Large jumps in step or angle are suppressed. Bounds are imposed to promote smooth trajectories and avoid jerky motion.
- *Environmental adaptability.* Bin sets can be reconfigured by scenario. Sparse static scenes require fewer bins. Highly dynamic scenes benefit from finer angular resolution or adaptive step sizing.

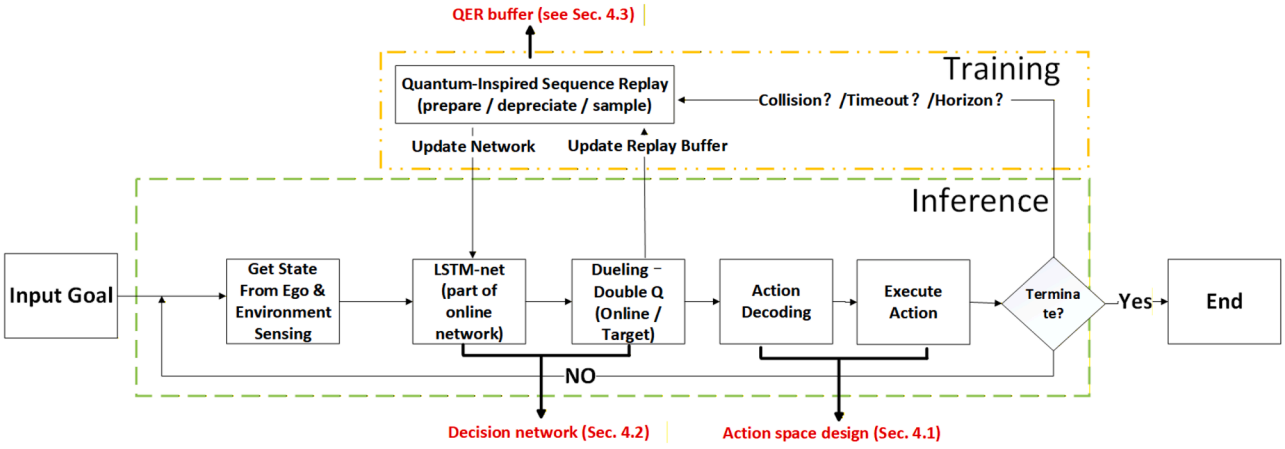


Fig. 1. A simplified pipeline of QER-LPD3QN.

Action set. Let S_{step} denote step-length bins and let Θ_{angle} denote heading-change bins. The action space is defined as the Cartesian product shown in Eq. (20):

$$\mathcal{A} = \left\{ a = (\Delta s, \Delta\theta) \ : \ \Delta s \in S_{\text{step}},\ \Delta\theta \in \Theta_{\text{angle}} \right\}. \tag{20}$$

A typical choice of bin sets is provided in Eq. (21):

$$S_{\text{step}} = \{\, 2.0,\ 5.0,\ 8.0,\ 10.0 \,\}, \qquad \Theta_{\text{angle}} = \{\, 0^\circ,\ 45^\circ,\ 90^\circ,\ \ldots,\ 315^\circ \,\}. \tag{21}$$

The total number of discrete actions follows directly from the product of the bin counts, as shown in Eq. (22):

$$|\mathcal{A}| = |S_{\text{step}}| \cdot |\Theta_{\text{angle}}|. \tag{22}$$

Smoothness constraints. To ensure smooth motion, per-step bounds on distance and heading rate are imposed. The instantaneous constraints are given in Eq. (23):

$$0 < \Delta s_t \le s_{\max}, \qquad |\Delta\theta_t| \le \theta_{\max}, \tag{23}$$

while optional rate limits are listed in Eq. (24):

$$|\Delta s_t - \Delta s_{t-1}| \le s^{\Delta}_{\max}, \qquad |\Delta\theta_t - \Delta\theta_{t-1}| \le \theta^{\Delta}_{\max}. \tag{24}$$

Here, $s_{\max}$ and $\theta_{\max}$ bound absolute command magnitudes. The terms $s^{\Delta}_{\max}$ and $\theta^{\Delta}_{\max}$ bound command rates. These limits are selected according to platform capabilities and safety considerations.

This discretized design allows flexible control of both distance and heading. It enables precise trajectory shaping in cluttered and dynamic environments while remaining compatible with value-based RL and batched training.

4.2. QER-LPD3Qn network architecture

LP-D3QN (LSTM-based Prioritized Dueling Double DQN) is adopted to address temporal decision making in path planning. Compared with a plain DQN, the input is sequence-structured. Temporal modeling is introduced by an LSTM. Dueling and Double designs are used to stabilize value estimation. Prioritization is attached at the replay stage.

4.2.1. Architecture overview

As shown in Fig. 2, the QER-LPD3QN network mainly consists of the following layers:

Input (sequence). Observations are flattened to vectors $\mathbf{x}_t \in \mathbb{R}^{d_s}$ per step. A sliding window of length L forms a sequence:

$$\mathbf{x} = \Big[\underbrace{\mathbf{x}_{\text{pos}}}_{2},\ \underbrace{\mathbf{x}_{\text{tgt}}}_{2},\ \underbrace{\mathbf{x}_{\text{stat-rel}}}_{2K_s},\ \underbrace{\mathbf{x}_{\text{dyn-rel}}}_{2K_d} \Big].$$

- $\mathbf{x}$ is the per-step observation vector. It is flattened.
- $\mathbf{x}_{\text{pos}} \in \mathbb{R}^2$: ego position features (e.g., x, y or $\Delta x, \Delta y$ in the ego frame).
- $\mathbf{x}_{\text{tgt}} \in \mathbb{R}^2$: target-relative features (e.g., $\Delta x, \Delta y$ or a two-dimensional polar encoding).
- $\mathbf{x}_{\text{stat-rel}} \in \mathbb{R}^{2K_s}$: concatenated relative positions of the K_s nearest static obstacles. Each obstacle contributes $(\Delta x, \Delta y)$.
- $\mathbf{x}_{\text{dyn-rel}} \in \mathbb{R}^{2K_d}$: concatenated relative positions of the K_{d} nearest dynamic obstacles. Each obstacle contributes $(\Delta x, \Delta y)$. Velocities can be appended if needed.

Dimension

$$d_s = \dim(\mathbf{x}) = 2 + 2 + 2K_s + 2K_d = 4 + 2\,(K_s + K_{\text{d}}).$$

Table 1
Network Components and Tensor Shapes.

Layer/Module	Input Tensor	Weight Shape	Output Tensor	Notes
Hidden Layer 1	$(B \cdot L,\ d_s)$	$d_s \times 256$	$(B \cdot L,\ 256)$	per-step linear
LSTM (1 layer)	$(B,\ L,\ 256)$	see note†	$(B,\ L,\ 256)$	`batch_first` = True
Hidden Layer 2	$(B,\ 256)$	256×64	$(B,\ 64)$	feature compression
fc_A	$(B,\ 64)$	64×32	$(B,\ 32)$	advantage head $A(s,\cdot)$
fc_V	$(B,\ 64)$	64×1	$(B,\ 1)$	value head $V(s)$
Dueling aggregation	$V,\ A$	—	$(B,\ 32)$	$Q = V + A - \text{mean}(A)$

LSTM layer. The LSTM is the core of the QER-LPD3QN architecture. The input state sequence is processed. Internal memory is maintained at each time step. Long-term dependencies are captured. In path planning, obstacle motion and environmental changes are temporally correlated. These patterns are learned by the LSTM. This enables anticipation of near-future planning decisions.

Fully connected layer. The LSTM output is fed into a fully connected layer. Higher-level features are extracted. With multiple layers, low-level features are abstracted into planning-relevant representations for the final decision.

Output layer. The output layer produces Q-values for all actions. The action with the highest Q-value is selected and returned to the planner. The Q-values determine the action to take in a given state to maximize the future return.

The parameters of each network component are listed in Table 1.

4.2.2. Role and design of the LSTM layer

The role of the LSTM in QER-LPD3QN is illustrated in Fig. 3. A length-L state sequence is encoded so that temporal dependencies and long-term context are preserved. In many classical planners, cross-time dependencies are not explicitly modelled. By contrast, internal memory is maintained by the LSTM and sequence regularities are captured in

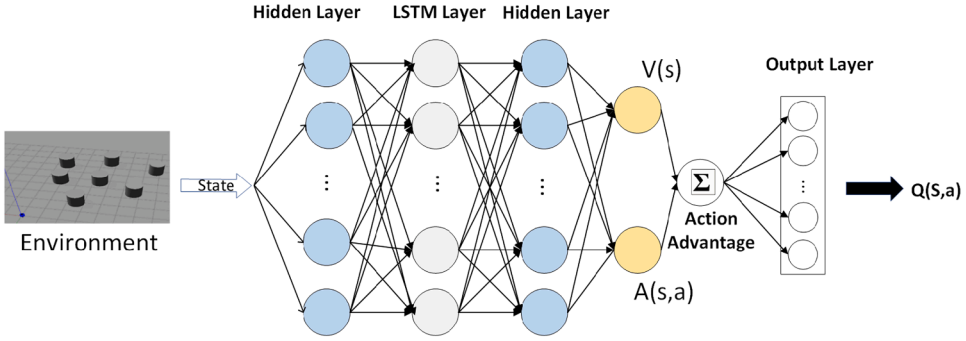


Fig. 2. Overview of the LP-D3QN architecture.

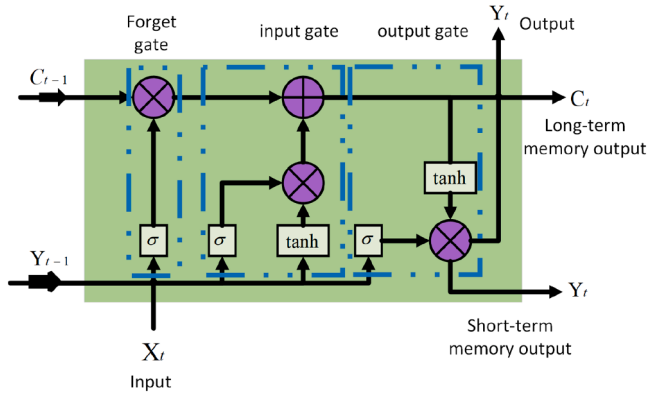


Fig. 3. Architecture of the LSTM module in LP-D3QN for encoding a length-L state sequence.

dynamic scenes. As a result, decision robustness can be improved under strong temporal variations.

The forget and input gates control memory. Useful history is retained. Irrelevant content is discarded. Past observations of obstacles and executed actions are thus exploited for decision making in dynamic environments.

The input gate adapts the internal state to new observations. Predicted motion of obstacles can be inferred from the sequence. Agaent actions are adjusted to avoid anticipated conflicts.

The output gate regulates the exposure of the memory to the decision head. Unstable responses are reduced. Local optima are less likely.

In summary, long-range dependencies are captured. Near-future environmental changes can be predicted from history. Proactive avoidance paths are planned. Accurate decisions are enabled in complex and dynamic scenes.

4.2.3. Output layer and Q-Value selection

After the LSTM processes the input sequence, the resulting temporal features are mapped to the action space through a fully connected layer. This output layer assigns a Q-value to every action in $\mathcal{A}$, following the standard definition of action-value functions in Q-learning. The action that maximizes this predicted return is selected at each step.

Specifically, for each state s_t, the network outputs the vector $\{Q_\theta(s_t, a)\}_{a\in\mathcal{A}}$. The index of the maximal element is returned, following the greedy rule defined later in Eq. (27). This implements the principle of selecting the action that maximizes the expected long-term reward.

To reduce overestimation bias and enhance discrimination between state values, Double DQN and a Dueling head are adopted. The online (evaluation) network selects the action, while the target network evaluates it. The one-step Double DQN target is computed using Eq. (25), shown below:

$$y_t = r_t + \gamma\, Q_{\bar{\theta}}\Big(s_{t+1},\ \arg\max_{a'\in\mathcal{A}} Q_\theta(s_{t+1}, a')\Big). \tag{25}$$

Let $|\mathcal{A}|$ denote the size of the action set. The Dueling decomposition produces the value $V_\theta(s)$ and advantage $A_\theta(s,a)$, which are combined according to Eq. (26):

$$Q_\theta(s,a) = V_\theta(s) + A_\theta(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'\in\mathcal{A}} A_\theta(s,a'). \tag{26}$$

The greedy action for state s_t is determined by Eq. (27):

$$a_t^* = \arg\max_{a\in\mathcal{A}} Q_\theta(s_t, a). \tag{27}$$

For sequence training, the TD target for sample (i,t) follows Eq. (28):

$$y_{i,t} = r_{i,t} + \gamma\,(1-d_{i,t})\, Q_\theta\Big(s'_{i,t}, a^*_{i,t}\Big). \tag{28}$$

The QER-LPD3QN replay mechanism then combines quantum-inspired rotations and depreciation updates to adjust sampling priorities and improve convergence efficiency.

4.3. Quantum-Inspired sequence experience replay buffer

To address the limits of conventional replay under temporal dependence in path planning, a Quantum-inspired Sequence Experience Replay Buffer (Q-ER) is proposed. Each experience is mapped to a quantum state. Sampling priorities are adjusted by quantum rotations and depreciation. The selection process is optimized. Temporal correlations in dynamic scenes are handled. The learning process is accelerated. Planning efficiency and accuracy are improved.

Fig. 4 summarizes the overall process of QER-LPD3QN. Two coupled loops are included. In the interaction loop, actions are selected and new experiences are collected and stored in the replay buffer. In the learning loop, mini-batches are sampled from the buffer and the agent is updated by backpropagation. The TD error and replay-frequency information are then fed back to update the sampling amplitudes.

Step 1: Quantum encoding. The experience transition produced by environment–agent interaction is denoted as $e_k = (s_t, a_t, r_t, s_{t+1})$. It is encoded as a quantum state $|\psi_k\rangle$ (see the Bloch-sphere illustration in Fig. 5).

Step 2: Prepare–initialization. A prepare operation is applied to each experience to obtain an initial superposition with amplitude. Its sampling probability is determined by the amplitude angle, $p_k = \sin^2(\theta_k/2)$.

Step 3: Composite buffer and sampling. All experience states form a composite system: $|\psi^{(\text{total})}\rangle = \bigotimes_k |\psi^{(k)}\rangle$. Mini-batches of sequences/transitions are drawn according to p_k and are used for one parameter update.

Step 4: Learning feedback. Backpropagation on the mini-batch updates the network. For each sample, the TD error δ_k and the replay count n_k are recorded.

Step 5: Amplitude update and depreciation. Angles θ_k are updated by a prepare operation using δ_k, and then depreciated using n_k. High-value

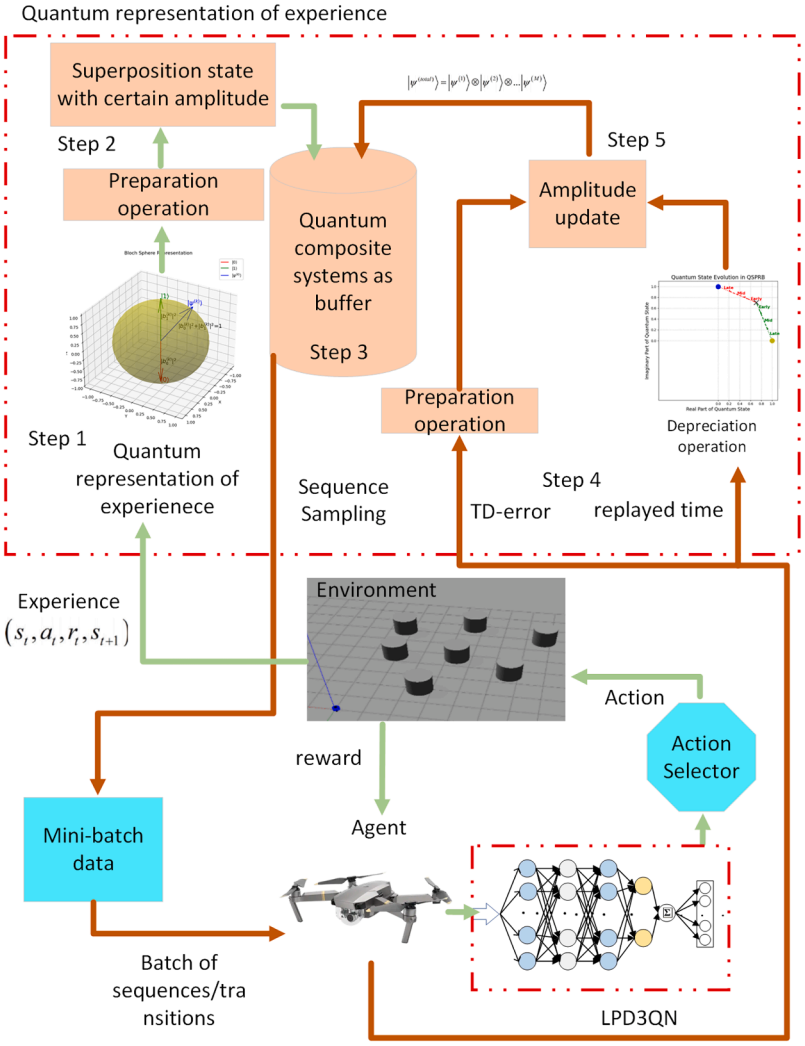


Fig. 4. Overall pipeline of QER-LPD3QN, including the interaction loop and the learning loop with the five main steps.

samples receive higher future sampling probability. Over-replay is suppressed. Updated quantum states are written back to the buffer.

4.3.1. *Visualization of the quantum mechanism*

To provide intuition, the evolution of quantum states in QER is visualized on the Bloch sphere. As shown in Fig. 5, a qubit state is expressed as defined in Eq. (29):

$$|\psi\rangle = b_0|0\rangle + b_1|1\rangle. \tag{29}$$

Here, $|0\rangle$ and $|1\rangle$ correspond to the north and south poles, respectively. Following the real-amplitude parameterization in Eq. (33), the amplitudes b_0 and b_1 are restricted to real numbers. As a result, the state evolution is confined to a great circle of the Bloch sphere.

In QER, each experience sequence is associated with a quantum state $|\psi\rangle$. The sampling priority is encoded in the amplitude of $|1\rangle$. Using Eq. (30), the sampling probability becomes

$$P = |b_1|^2 = \sin^2(\theta/2), \tag{30}$$

where θ denotes the polar angle between $|\psi\rangle$ and $|0\rangle$.

At initialization, all sequences are assigned the uniform superposition defined in Eq. (31):

$$|\psi_0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad \Rightarrow \quad P = .5, \tag{31}$$

which implements uniform sampling and places the state on the equator of the Bloch sphere.

During training, the state is updated by two rotation operations. When an experience exhibits a large TD error, the *prepare* operator rotates its state toward $|1\rangle$, following the rotation rule in Eq. (37). This increases $|b_1|^2$ and therefore raises the sampling probability. In contrast, when an experience has been replayed many times, the *depreciate* operator rotates its state toward $|0\rangle$, reducing $|b_1|^2$ and suppressing further sampling. This geometric interpretation is illustrated in Fig. 5.

The accumulated effect of these rotations is visualized in Fig. 6. High-value experiences (High-value / Prepare, red trajectory) move rapidly from the equator toward $|1\rangle$. Low-value experiences (Low-value / Depreciate, green trajectory) drift toward $|0\rangle$. Through this dynamic adjustment, the replay buffer concentrates on the most informative sequences, while over-replay of redundant experiences is avoided.

4.3.2. *Quantum-State representation*

In QER, the priority of each experience sequence is represented as a quantum state, following the fundamental qubit definition in quantum computation Wei et al. (2021). As defined in Eq. (32), we have:

$$|\psi\rangle = b_0|0\rangle + b_1|1\rangle, \qquad |b_0|^2 + |b_1|^2 = 1. \tag{32}$$

where b_0 and b_1 are complex probability amplitudes satisfying the normalization condition. In our implementation, to simplify computation and improve efficiency, b_0 and b_1 are restricted to real numbers. Consequently, the quantum state corresponds to a point on the great

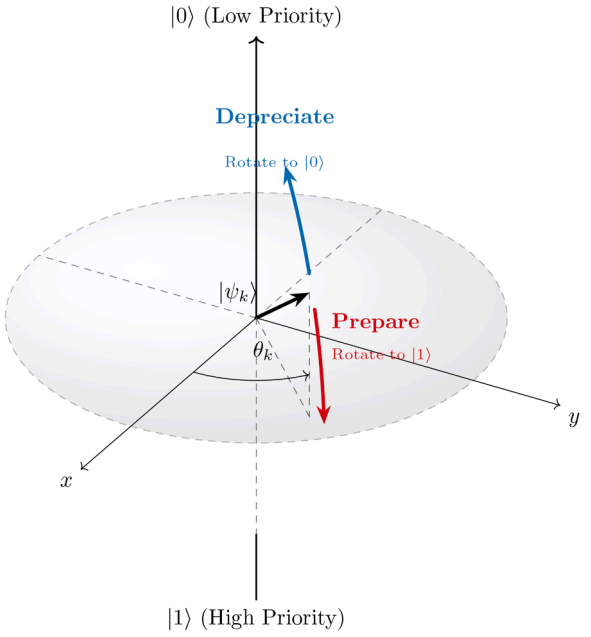


Fig. 5. Geometric representation of QER on the Bloch sphere. The prepare and depreciate operations are illustrated as rotations of $|\psi_k\rangle$ toward $|1\rangle$ and $|0\rangle$, respectively.

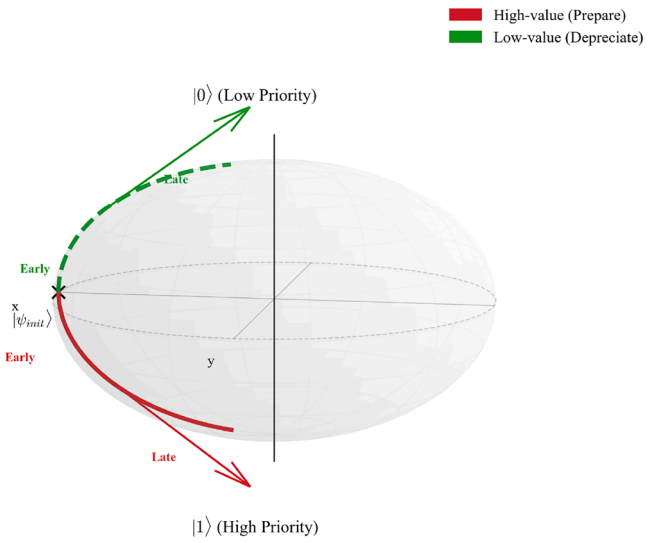


Fig. 6. Trajectory evolution on a Bloch-sphere great circle.

circle of the Bloch sphere, parameterized as defined in Eq. (33):

$$b_0 = \cos\frac{\theta}{2}, \qquad b_1 = \sin\frac{\theta}{2}, \qquad \theta \in [0, \pi]. \tag{33}$$

Here, θ is the polar angle determining the probability distribution. This real-valued parameterization offers several specific advantages for real-time path planning:

- *Computational efficiency.* Restricting amplitudes to real numbers avoids complex arithmetic operations, significantly reducing the computational cost during backpropagation.
- *Reasonable dynamic range.* The mapping $\theta \in [0, \pi]$ yields a sampling probability P that varies smoothly from 0 to 1, covering the full priority spectrum.
- *Sequence adaptivity.* The priority is derived from sequence-wise averaging, which effectively preserves temporal coherence compared to independent transition sampling.

The squared amplitude $|b_1|^2$ directly determines the sampling probability, consistent with the quantum measurement postulate.

At initialization, all experience sequences are assigned the uniform superposition state, as defined in Eq. (34):

$$|\psi_0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \tag{34}$$

which corresponds to $\theta = \pi/2$. In this state, the sampling probability is $P = |b_1|^2 = 0.5$, resulting in uniform sampling before learning begins. As training proceeds, the quantum state is dynamically adjusted by the *prepare* and *depreciate* operations, allowing the sampling policy to adaptively track the learning process.

To prevent the dilution of inter-sample priority differences, no unitary transform is applied to the composite system as a whole; instead, a specific rotation (analogous to the Grover operator) is applied to each individual experience sequence.

4.3.3. *Prepare operation*

A core mechanism of the Quantum-Inspired Sequence Experience Replay (QER) is the dynamic adjustment of priorities based on the TD error of experience sequences. This ensures that the sampling probability of high-value experiences is increased, thereby accelerating learning. This objective is achieved via the *prepare* operation using quantum rotation gates, which is formally defined in Eqs. (35)–(39).

A preparation factor σ is defined to control the granularity of the quantum state rotation. It encodes the influence of the current training episode (TE) on the rotation magnitude. As training proceeds, σ is annealed to reduce the rotation step size, stabilizing convergence. The decay schedule is defined in Eq. (35):

$$\sigma = \frac{\zeta_1}{1 + e^{\zeta_2/\mathrm{TE}}}, \tag{35}$$

where ζ_1 and ζ_2 are hyperparameters. Under this schedule, σ decreases with TE and asymptotically approaches a constant (e.g., $\zeta_1/2$), bounding the effective rotation in later training stages.

We employ a priority-driven rotation rule to dynamically determine the update step. The priority of a sequence is given by $P_k = |\delta_k| + \varepsilon$, normalized by the buffer-wide maximum $P_{\max} = \max_j P_j$. The number of rotation steps, denoted as m_k, is calculated based on the relative priority as shown in Eq. (36):

$$m_k = \left\lfloor \mu \cdot \frac{P_k}{P_{\max}} - \frac{\iota}{\sigma} \right\rfloor, \tag{36}$$

where $\mu, \iota > 0$ are scaling constants. Positive values of m_k induce a rotation toward the high-amplitude pole $|1\rangle$ (prepare), while negative values rotate toward $|0\rangle$ (depreciate). We clip m_k to prevent excessively large updates. Crucially, m_k is proportional to P_k, ensuring that higher-priority experiences receive larger angular adjustments.

Given the rotation steps m_k, the quantum state amplitudes are updated by applying the rotation matrix defined in Eq. (37):

$$\begin{pmatrix} b_0^{(k)} \\ b_1^{(k)} \end{pmatrix} = \begin{pmatrix} \cos(m_k\sigma) & -\sin(m_k\sigma) \\ \sin(m_k\sigma) & \cos(m_k\sigma) \end{pmatrix} \begin{pmatrix} b_0^{(k)} \\ b_1^{(k)} \end{pmatrix}. \tag{37}$$

This operation adjusts the state's direction on the Bloch sphere. When $m_k > 0$, the state rotates toward $|1\rangle$, increasing the sampling probability defined by $|b_1|^2$. Conversely, when $m_k \le 0$, it rotates toward $|0\rangle$.

Mathematically, this discrete update corresponds to applying a basic unitary rotation operator U_σ, which is equivalent to the rotation gate $R_y(-2\sigma)$ in quantum computing Yu et al. (2024):

$$U_\sigma = e^{-i\sigma Y} = \begin{bmatrix} \cos(\sigma) & -\sin(\sigma) \\ \sin(\sigma) & \cos(\sigma) \end{bmatrix}. \tag{38}$$

Consequently, the total transformation is the result of applying U_σ iteratively m_k times:

$$U_\Phi = (U_\sigma)^{m_k}. \tag{39}$$

4.3.4. Depreciation operation

The depreciation operation reduces the priority of over-sampled or well-learned experiences, preventing the replay buffer from being dominated by a small subset of transitions. This mechanism maintains a balance between exploration and exploitation over time. The update rule is defined by the rotation in Eq. (40):

$$\begin{pmatrix} b_0' \\ b_1' \end{pmatrix} = \begin{pmatrix} \cos\omega & \sin\omega \\ -\sin\omega & \cos\omega \end{pmatrix} \begin{pmatrix} b_0 \\ b_1 \end{pmatrix}. \tag{40}$$

The depreciation factor ω determines the rotation magnitude toward the low-amplitude pole $|0\rangle$. Its value is defined in Eq. (41):

$$\omega = \frac{\tau_1}{N_k\left(1 + e^{\tau_2/\mathrm{TE}}\right)}, \tag{41}$$

where N_k is the number of times sequence k has been sampled, TE is the current training episode, and $\tau_1, \tau_2 > 0$ are hyperparameters.

This formulation has several key properties:

- *Inverse dependence on sampling frequency.* As N_k increases, ω decreases, slowing depreciation and preventing premature removal of useful transitions.
- *Adaptation to training progress.* In early training (small TE), ω is relatively large, allowing faster adjustment. In later stages, ω becomes smaller, ensuring stable priorities.
- *Clear geometric interpretation.* The rotation in Eq. (40) moves the state toward $|0\rangle$, reducing the sampling probability $P = |b_1|^2$.

4.3.5. Sequential sampling mechanism

Temporal adjacency is disrupted by single-step replay. The modeling of dynamic obstacles and motion trends is thereby weakened. To preserve temporal consistency in path decision-making, sequence-level modeling is adopted in QER.

Contiguous transition sequences of length L are used as the minimal sampling unit. Each sequence is associated with a qubit-like quantum state $|\psi_k\rangle = b_0^{(k)}|0\rangle + b_1^{(k)}|1\rangle$. The probability of being selected into a minibatch is determined by the squared amplitude of this state, $P_k = |b_1^{(k)}|^2$. Both high-value focus and temporal structure are preserved. Sampling a sequence is equivalent to measuring its quantum state, which causes wavefunction collapse.

To handle non-stationarity in online training, the sampled sequence is reset to a uniform quantum state. The latest TD error and replay count are then applied to this state through *prepare* and *depreciation* operations, respectively. The updated state is written back to the replay buffer. An "observe-prepare-depreciate" cycle is thus formed.

Let the k-th sequence be composed of $\{e_t\}_{t=\tau}^{\tau+L-1}$, where each transition $e_t = (s_t, a_t, r_t, s_t', d_t)$ is associated with a quantum state $|\psi_t\rangle = b_{0,t}|0\rangle + b_{1,t}|1\rangle$. The importance of the sequence is characterized by the average of the member probabilities. The sequence sampling probability is defined in Eq. (42):

$$\overline{P}_k = \frac{1}{L}\sum_{t=\tau}^{\tau+L-1} \left|b_{1,t}\right|^2, \tag{42}$$

where $|b_{1,t}|^2$ denotes the measurement probability of accepting that experience. Decision segments marked as important across multiple consecutive timesteps are thus preferentially selected.

During sampling, strict boundary checks are performed. Overlapping sequences of length L are constructed only within the same episode. This prevents information leakage and temporal discontinuities across episode boundaries. For segments near terminal states with insufficient length, padding is applied via a mask.

To align with the temporal encoding mechanism of LSTM, QER constructs a batch-sequence tensor of shape (B, L) during sampling, where B is the batch size and L is the sequence length. The sample of the i-th sequence at timestep t is denoted in Eq. (43):

$$(s_{i,t}, a_{i,t}, r_{i,t}, s_{i,t}', d_{i,t}), \quad i = 1, \ldots, B,\ t = 1, \ldots, L. \tag{43}$$

The corresponding TD residual is given by Eq. (44):

$$\delta_{i,t} = y_{i,t} - Q_\theta(s_{i,t}, a_{i,t}), \tag{44}$$

where the computation of $y_{i,t}$ is detailed in Section 4.2.3.

After importance sampling weights $w_{i,t}$ are introduced, the weighted sequence loss is defined in Eq. (45):

$$\mathcal{L}_{\mathrm{MSE}}(\theta) = \frac{1}{\sum_{i,t} m_{i,t}} \sum_{i=1}^{B}\sum_{t=1}^{L} m_{i,t}\, w_{i,t}\, \delta_{i,t}^2, \tag{45}$$

where $m_{i,t} \in \{0, 1\}$ is used to mask padded positions and episode boundaries.

From an implementation perspective, the replay buffer stores only atomic state transitions $e_t = (s_t, a_t, r_t, s_t', d_t)$. For each episode, a time-ordered index list is maintained to mark episode boundaries.

Candidate sequences of length L are constructed by sliding a window over this index list. Thus, the transitions within each sequence are temporally contiguous and consistent with the original interaction order. In practice, each window is represented only as an index interval. No independent data structure is created for the sequence. Overlapping sequences share the same underlying transitions and their quantum parameters. Sequence construction is strictly confined within a single episode. Cross-episode windows are explicitly prohibited at the code level.

Quantum states are maintained at the transition level, not at the sequence level. Each atomic transition e_t is associated with a set of real-valued quantum amplitudes $(b_{0,t}, b_{1,t})$ and a replay counter N_t. Consequently, the quantum state of a sequence is implicitly defined as the average of the member transition probabilities. The sequence sampling probability $\overline{P}_k$ in Eq. (42) is computed online from its members $\{b_{1,t}\}_{t=\tau}^{\tau+L-1}$ during sampling. No additional sequence-level state is stored in the buffer.

When a sequence is sampled and used for network updates, the TD residual signal is applied to all member transitions within the window. Their replay counters are incremented. The quantum amplitudes are updated in place via two operators: *prepare*, driven by TD error, and *depreciation*, driven by replay count.

Because overlapping sequences share the same per-transition quantum parameters, critical transitions appearing in multiple high-value sequences are consistently reinforced. Conversely, over-replayed transitions are gradually down-weighted through depreciation as their replay counts increase. Thus, temporal context is preserved while the risk of redundant updates from overlapping windows is mitigated.

The full sequence sampling procedure is summarized in Algorithm 1.

One sampling iteration (pseudo-workflow).

1. *Probability evaluation.* Compute sequence probabilities $\{p_k\}$ in the buffer.
2. *Normalization and draw.* Normalize and sample N sequences without replacement (roulette-wheel).
3. *Temporal consistency.* Feed sequences to the LSTM encoder in the original order; apply masks for stability.
4. *Training and updates.* Update the network; for all member transitions, run prepare (by TD error) and then depreciation (by replay count); write back updated quantum states.
5. *Window advance.* Newly formed sequences are initialized to the uniform state; FIFO and capacity limits are enforced.

4.3.6. Summary: The QER sequential replay mechanism and its relationship to PER

The QER sequential experience replay mechanism is briefly summarized in this subsection. A comparison with the classic prioritized experience replay (PER) is provided, and the relationship between the two methods is clarified by directly referencing the mathematical definitions introduced in Section 4.3.2.

Algorithm 1 QER Sequence Sampling (Observe-Prepare-Depreciate).

Input: mini-batch B, sequence length L, draws N, IS exponent β, probability floor ε, masks m_t, buffers D
Output: mini-batch and IS weights $\{w_j\}$
Build candidates: slide a window of length L within each episode; form C; set $M \leftarrow |C|$
for each sequence $S \in C$ **do**
  compute $p(S) \leftarrow \max\left(\text{mean}_t(m_t\, b_{1,t}^2),\, \varepsilon\right)$
end for
normalize $\{p(S)\}$ to probabilities probs
sample N sequences without replacement by probs; store indices in idx
pack tensors in temporal order; compute per-step TD residuals δ_j
compute IS: $w_j \leftarrow \left(M \cdot \text{probs}[\text{idx}_j] + \varepsilon\right)^{-\beta}$; normalize $\{w_j\}$
loss: $\mathcal{L} \leftarrow \text{mean}_j\left[w_j \cdot \text{Huber}(\delta_j)\right]$; apply gradient clipping
for each sampled sequence S_j **do**
  Observe: collapse its state to obtain the outcome
  Reset: set sequence state to the uniform state
  Prepare: apply rotation by current TD error; update amplitudes
  Depreciate: apply decay by replay count; update amplitudes
  renormalize the sequence state; write back to D
end for

PER is one of the most widely used experience replay methods. Experience is stored as single-step transitions, formally denoted as

$$e_t = (s_t,\, a_t,\, r_t,\, s_{t+1}). \tag{46}$$

Each transition is assigned a scalar priority based on its temporal-difference (TD) error,

$$p_t = |\delta_t| + \epsilon, \tag{47}$$

where ϵ is a small positive constant ensuring non-zero probability. This mechanism accelerates the utilization of "hard-to-learn" transitions. However, trajectories are fragmented into independent samples, making temporal structure difficult to preserve and the effect of replay frequency only implicitly expressed through buffer dynamics.

The proposed QER mechanism extends the PER framework in three key aspects. First, the sampling unit is upgraded from isolated transitions to short sequences. Temporal continuity of dynamic obstacle motion and local topological structure is better preserved, and compatibility with LSTM-based modeling is naturally achieved. Second, each sequence is associated with a qubit-like state whose structure is defined in Eq. (32) and parameterized in Eq. (33). The amplitude of this state determines its sampling probability through the measurement rule, $P_k = |b_1^{(k)}|^2$, and its sequence-level version is formalized in Eq. (42). Third, the amplitude is dynamically updated through the *prepare* and *decay* rotations, whose rotation schedules are determined by Eq. (35) and Eq. (36), and whose state update rule is expressed in Eq. (37). Thus, both TD error and replay count are explicitly encoded into the qubit amplitudes, yielding joint characterization of "information content" and "usage history." A detailed comparison is presented in Table 2.

As shown in Table 2, QER inherits the core PER idea of emphasizing transitions with high TD error. However, the following three extensions address major limitations of the PER baseline.

First, extending the sampling unit from transitions to sequences enables replay of local motion patterns and safety-relevant temporal dependencies as coherent structures.

Second, replacing scalar priorities with amplitude-based priorities (Eqs. (32)-(33)) allows TD-error-driven amplification via the *prepare* operator and replay-count-driven attenuation via the *decay* operator (Eqs. (35)-(37)). The sampling probability P_k is thereby shaped by both informativeness and replay frequency. ss Third, the rotation angles controlling prepare and decay follow a training-epoch schedule (Eq. (35)), enabling coarse exploration in early training and a sharper, more selective distribution in later stages.

Table 2
Comparison Between PER and QER.

Aspect	PER (Baseline)	QER (Ours)
Sampling unit	Single-step transitions are used.	Short sequences (sliding windows of length L within trajectories) are used.
Priority representation	Scalar weights are assigned based on TD errors.	Qubit-like state amplitudes are maintained. TD error and replay count are jointly encoded.
Temporal structure	Trajectories are fragmented. Temporal structure is lost.	Temporal continuity within sequences is preserved.
Replay-frequency handling	Replay frequency is implicitly reflected via buffer dynamics.	Over-replayed sequences are explicitly suppressed through decay rotations.
Training adaptability	Priority formulas are fixed. Only a few global hyperparameters are adjusted.	Rotation angles are scheduled over training epochs. The sampling distribution is progressively sharpened.

Notes: PER–Prioritized Experience Replay; QER–Quantum-Inspired Experience Replay.

It should be emphasized that the "quantum-inspired" aspect of QER is characterized by qubit-like probability amplitudes and Bloch-sphere rotation operators, which have been widely adopted in quantum-inspired optimization and learning frameworks Olvera et al. (2023), Yu et al. (2024). The proposed mechanism is implemented in a fully classical manner, and no quantum hardware is required Wei et al. (2021), Yu et al. (2024). The benefit is mainly achieved by reshaping sampling probabilities and by incorporating replay-frequency information into priority updates, which has been reported to improve learning efficiency and stability in quantum-inspired experience replay Wei et al. (2021). Different from transition-level designs, replay priorities are assigned at the sequence level to explicitly capture temporal dependencies in dynamic path planning.

Chapter 5 provides comparative and ablation experiments using a unified network architecture and identical training hyperparameters. Performance differences between PER and QER are empirically quantified, complementing the mechanistic analysis presented in this section.

5. Experiments and results

The feasibility of QER-LPD3QN was assessed in simulation. Implementations were built in Visual Studio Code. Four additional planners were included for comparison. Four 150×150 visual environments were created under the `gym` framework. Hyperparameters of QER-LPD3QN are listed in Table 3. Training was executed on Windows 11 with an Intel Core Ultra 7 265K (3.90 GHz) CPU and an NVIDIA RTX 5070 Ti GPU. Under identical unknown-obstacle settings, QER-LPD3QN was compared with all baselines. Superiority over IDDQN and others was observed. Robustness under random disturbances in unknown environments was preliminarily evaluated.

5.1. Comparison with baselines

The algorithms and configurations are summarized below.

1. **IDDQN.** Action values are estimated with twin networks (online and target). Experience replay and soft target updates are applied for stability. A dense reward combines arrival/distance, boundary violation, and safety clearance. Exploration-exploitation is guided and constrained. Local policies are learned without a prior map Bai et al. (2024).

Table 3
Hyperparameters used in experiments.

Name of parameters	Value
Action space size	8
Learning rate α	0.0025
Decay factor γ	0.9
Start ε-greedy value	0.9
Final ε-greedy value	0.01
Exploration decay rate	500
Experience replay pool	10,000
Mini-batch size	64
Target network update step	4
Hidden layers size	2
Number of neurons	128
Activation function	ReLU
$(\lambda_1, \lambda_2, \lambda_3, \lambda_4)$	$(1, -0.35, -1, -1)$

Notes: α–learning rate; γ–discount factor; ε–ε-greedy exploration schedule.

2. **RRT*.** An asymptotically optimal sampling-based planner. Random samples expand a tree with nearest-neighbor connections. Rewiring shortens the path. Optimality is approached as samples grow. Time-limited sampling and periodic re-planning are used for real-time constraints. Obstacles are inflated during collision checks. The path is converted to continuous control commands.
3. **APF.** A potential field is formed by goal attraction and obstacle repulsion. Descent along the negative gradient is performed. Computation is light and implementation is simple. Local minima and corridor oscillations may occur. A distance-threshold decay is applied to repulsion. Mild random perturbation/escape is used. In dynamic scenes, the field is updated per frame for online avoidance.
4. **DWA-like.** Candidate velocities are sampled in the feasible window of (v, ω) under kinodynamic and acceleration limits. Short-horizon forward simulation is performed for each candidate. A composite cost scores goal heading, minimum obstacle distance, and speed/arrival terms. The best control is executed. Dynamic obstacles and receding-horizon optimization are naturally supported. Real-time performance and collision-free motion within the prediction window are achieved.
5. **A*.** A heuristic shortest-path search on a grid or sparse graph. Consistent heuristics (Euclidean/Manhattan) provide feasibility and an optimality bound. Obstacles are morphologically inflated to meet footprint and safety margins. Fixed-rate re-planning is used in dynamic scenes. The discrete path is smoothed and converted to a continuous reference trajectory.

All methods were evaluated under identical hardware, simulation step size, map resolution, obstacle inflation, re-planning frequency, and control period. Observation noise and latency were matched. Public reference implementations were adopted when available. Results are reported as mean±standard deviation over five random seeds.

Implementation notes. Results are averaged over $n = 5$ random seeds; we report mean±std in this paper.

5.1.1. Static-Environment comparison

Two static maps were designed to test performance under stationary obstacles: (1) *dense obstacles*, and (2) a *narrow cross intersection*. We evaluate the four metrics introduced in this section: the minimum obstacle clearance M_d, the average clearance A_d, the planning time T, and the path length L. Fig. 7 visualizes trajectories in two dense-obstacle maps; aggregated results are reported in Table 4 and Table 5 (units: M_d/A_d in m, T in ms, L in m).

All planners reached the goal in both dense-obstacle trials. QER-LPD3QN achieved a mean planning time of 11.42 ms, faster than A* (75 ms), RRT* (302 ms), APF (22 ms), and DWA-like (57 ms). While classical planners sometimes produced slightly shorter paths, they were slower and tended to stay closer to obstacles.

5.1.1.1. Scenario 1 (dense obstacles v1).

All six methods succeeded without collision (*success* = True, *collides* = False).

- **Real-time performance.** QER-LPD3QN: $T = 11.33$ ms. It is close to IDDQN (9.52 ms) and markedly faster than A* (76.90 ms; ≈ 85% lower) and RRT* (501.74 ms; ≈ 97.7% lower). Learning-based planners thus show a clear latency advantage for online use.
- **Safety margins.** The minimum clearance of QER-LPD3QN is 0.6907, the best among non-IDDQN methods. The average clearance is 7.93, higher than A* (6.17), RRT* (6.91), APF (5.07), and DWA-like (6.47). Compared with DWA-like (0.1454), the minimum gap is about 4.7× larger. Compared with A* (≈ 0), the gain is pronounced.
- **Smoothness.** QER-LPD3QN attains 0.02, tied with RRT* (0.02), and better than A* (0.08) and APF (0.09). Millisecond-level latency is achieved without compromising trajectory quality.
- **Path length.** QER-LPD3QN: $L = 175.00$. It is slightly longer than A* (171.27) and APF (170.03), but shorter than RRT* (184.93) and IDDQN (185.00). In short, a marginally longer path trades for larger safety margins, better smoothness, and lower latency.

Overall, QER-LPD3QN balances *clearance* ↑, *smoothness* ↑, and *latency* ↓. It outperforms classical planners on key safety-timeliness criteria. Versus IDDQN, it is slightly slower but yields a shorter and smoother path.

5.1.1.2. Scenario 2 (dense obstacles v2).

All methods except APF avoided collision. APF reports *min_clearance* = 0 with *collides* = True, and is treated as a failure case.

- **Real-time performance.** QER-LPD3QN: $T = 16.15$ ms, second only to IDDQN (11.05 ms). Latency is ≈ 77.7% and ≈ 85.5% lower than A* (72.37 ms) and RRT* (111.58 ms), respectively. The online advantage is confirmed.
- **Safety margins.** RRT* attains the highest minimum clearance (3.60). QER-LPD3QN ranks second (2.77), and is higher than A* (0.74) and DWA-like (0.07). For average clearance, QER-LPD3QN and RRT* both reach 8.82, second only to IDDQN (9.41). Against A*, the minimum gap is ∼ 3.7× larger.
- **Smoothness.** RRT* is best (0.01). QER-LPD3QN achieves 0.03, better than DWA-like (0.04) and IDDQN (0.07), and close to A* (0.02). Thus, speed is gained while maintaining trajectory quality.
- **Path length.** QER-LPD3QN: $L = 170.00$. It is near the shortest (A*: 167.27, RRT*: 167.95) and clearly shorter than IDDQN (190.00). Hence, a near-shortest path is achieved together with strong safety margins.

In summary, with APF failing, QER-LPD3QN attains millisecond-scale latency, average clearance on par with RRT*, the second-best minimum clearance, and a near-shortest path with good smoothness. Compared with IDDQN, paths are substantially shorter and smoother.

QER-LPD3QN achieves millisecond-level planning, larger clearances than A*/DWA-like/APF, and smoothness comparable to or better than RRT*. Compared with IDDQN, it attains shorter and smoother paths. These gains align with the method's design: quantum-inspired replay that prioritizes informative sequences while depreciating over-sampled ones, and an LSTM-Dueling Double DQN backbone that improves sample efficiency without inducing aggressive action sequences that erode safety margins.

Two turn cases were evaluated: left turn and right turn. Results are shown in Fig. 8, Table 6, and Table 7.

In left turn.All methods except APF reached the goal. Among classical planners, A* and DWA-like produced the shortest paths (86.52 and 86.84), but the minimum clearances were only 0.50 and 0.47, indicating low safety margins. RRT* achieved strong safety and smoothness (min = 1.37, avg = 5.77, smoothness = 0.01), but required 60.36 ms. Among learning methods, QER-LPD3QN was the fastest (6.26 ms). It tied RRT* on smoothness (0.01) and reached min = 1.31, avg = 4.47. Overall, it outperformed A* and DWA-like and approached RRT*/IDDQN.

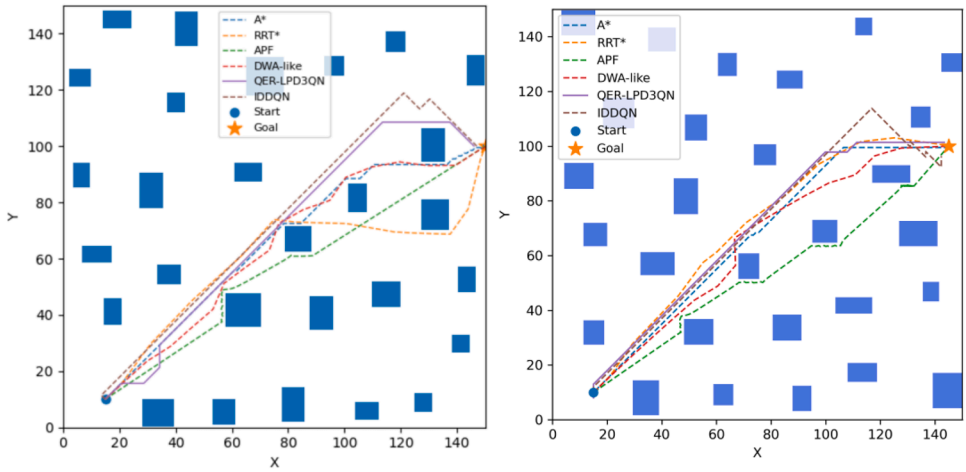


Fig. 7. The effect of dense static obstacle planning.

Table 4
Scenario 1 path planning results.

Algorithm	Success	Collides	Path length(m)	Min clearance(m)	Avg clearance	Path smoothness	Planning time (ms)
A*	True	False	171.27 ± 1.71	0.0001 ± 0.0000	6.17 ± 0.06	0.08 ± 0.005	76.90 ± 1.54
RRT*	True	False	184.93 ± 1.85	0.2915 ± 0.0583	6.91 ± 0.07	0.02 ± 0.005	501.74 ± 10.03
APF	True	False	170.03 ± 5.10	0.0091 ± 0.0018	5.07 ± 0.15	0.09 ± 0.005	24.04 ± 0.48
DWA-like	True	False	172.67 ± 5.18	0.1454 ± 0.0291	6.47 ± 0.19	0.04 ± 0.005	64.88 ± 1.30
QER-LPD3QN	True	False	175.00 ± 2.62	0.6907 ± 0.1381	7.93 ± 0.12	0.02 ± 0.005	11.33 ± 0.23
IDDQN	True	False	185.00 ± 3.70	2.5483 ± 0.5097	9.61 ± 0.19	0.03 ± 0.005	9.52 ± 0.19

Metrics: path_length (path length), min/avg_clearance (minimum/average clearance), path_smoothness (trajectory smoothness), planning_time_ms (planning time in milliseconds). Values are reported as mean ± standard deviation over repeated trials.

Table 5
Scenario 2 path planning results.

Algorithm	Success	Collides	Path length(m)	Min clearance(m)	Avg clearance	Path smoothness	Planning time (ms)
A*	True	False	167.270 ± 1.673	0.740 ± 0.148	8.070 ± 0.081	0.020 ± 0.005	72.373 ± 1.447
RRT*	True	False	167.950 ± 1.680	3.600 ± 0.720	8.820 ± 0.088	0.010 ± 0.005	111.578 ± 2.231
APF	True	True	168.190 ± 5.046	0.000 ± 0.050	1.300 ± 0.390	0.020 ± 0.005	35.230 ± 0.705
DWA-like	True	False	169.010 ± 5.070	0.070 ± 0.014	6.870 ± 0.206	0.040 ± 0.005	57.966 ± 1.159
QER-LPD3QN	True	False	170.000 ± 2.550	2.770 ± 0.554	8.820 ± 0.132	0.030 ± 0.005	16.148 ± 0.323
IDDQN	True	False	190.000 ± 3.800	2.050 ± 0.410	9.410 ± 0.188	0.070 ± 0.005	11.045 ± 0.221

Values are mean ± standard deviation across repeated trials.

IDDQN attained the highest clearances (min = 1.68, avg = 4.70), but had the poorest smoothness (0.09) and a longer path (97.00). APF failed in this case, with near-zero clearance, a much longer path (144.00), and 198.31 ms, revealing instability.

In the right-turn case, trends were similar. All methods except APF succeeded without collision. RRT* again excelled in mean clearance and smoothness (avg = 5.41, smoothness = 0.02), but had a large delay (86.92 ms). A* and DWA-like remained short in length (86.79 and 87.09), but their minimum clearances were only 0.50 and 0.44, posing wall-hugging risks. IDDQN reached 5.47 ms, but had min = 1.02 and smoothness = 0.08; its trajectory quality lagged behind QER-LPD3QN. APF was fast (3.24 ms) but unsafe (min = 0.0002) with poor smoothness (0.30). QER-LPD3QN delivered the best minimum clearance (3.00) and the best smoothness (0.01). It maintained high safety margins while keeping the path length at 90.00 and the planning time at 8.77 ms.

In this constrained intersection, QER-LPD3QN sustained millisecond-level latency. It ranked at or near the top in smoothness and safety margins, while keeping lengths near the shortest. RRT* offered high trajectory quality but excessive runtime, limiting real-time use. A* and DWA-like were short but risk-prone due to low clearances. IDDQN was fast but produced rougher trajectories. APF was unstable. Overall, QER-LPD3QN provided the most balanced trade-off among real-time performance, safety, and trajectory quality.

5.1.2. *Dynamic-Environment comparison*

To thoroughly validate the robustness and temporal reasoning capabilities of the proposed method, two distinct dynamic scenarios were designed: the high-difficulty “Rotating Broom” setting and a complex “Dynamic Traffic Flow” scenario. It is noteworthy that classical planning methods (A*, RRT*, APF, DWA) were extensively tuned but failed to adapt to these highly dynamic settings due to their quasi-static assumptions. Therefore, the following analysis focuses exclusively on learning-based approaches, comparing QER-LPD3QN against strong baselines including TD3, PPO, IDDQN, and a sequence-aware variant of Prioritized Experience Replay (PER_LSTM).

5.1.2.1. Rotating Broom Scene. This scenario contains rigid arms rotating at fixed angular speeds. Continuous swept regions are formed. Explicit temporal prediction is required. Representative trajectories are shown in Fig. 9a. QER-LPD3QN selects a safe upper passage and yields a smooth path. By contrast, PPO and PER_LSTM produce longer routes. TD3 tends to detour. IDDQN exhibits oscillatory steering in this scene.

As detailed in Table 8, safety is the primary differentiator. QER, IDDQN, TD3, and PER_LSTM all achieved a **100% success rate**. In contrast, the on-policy algorithm PPO failed to reliably navigate the dynamic obstacles, yielding only an 80% success rate. While PPO recorded

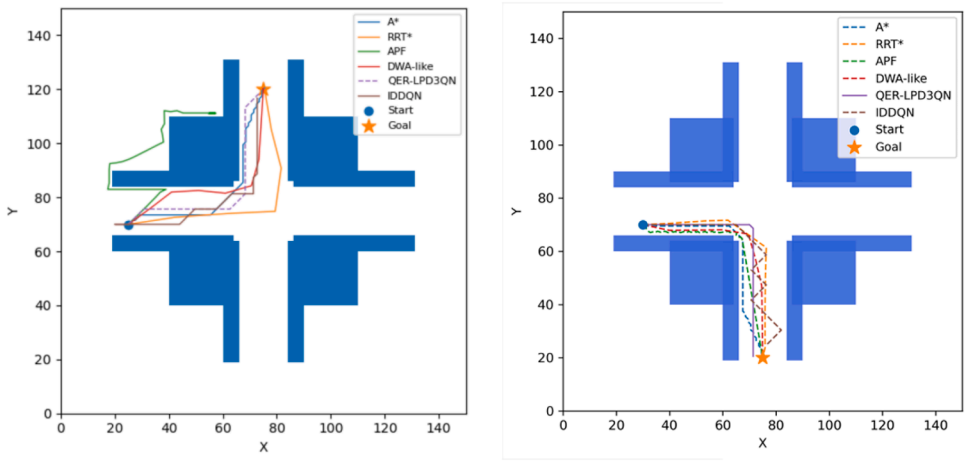


Fig. 8. The effect of narrow crossroads Planning.

Table 6
Scenario: Left-turn narrow intersection – path planning results.

Algorithm	Success	Collides	Path length(m)	Min clearance(m)	Avg clearance	Path smoothness	Planning time (ms)
A*	True	False	86.520 ± 0.865	0.500 ± 0.100	4.220 ± 0.042	0.030 ± 0.005	25.250 ± 0.505
RRT*	True	False	100.790 ± 1.008	1.370 ± 0.274	5.770 ± 0.058	0.010 ± 0.005	60.360 ± 1.207
APF	False	False	144.000 ± 4.320	0.020 ± 0.004	0.050 ± 0.015	0.020 ± 0.005	198.310 ± 3.966
DWA-like	True	False	86.840 ± 2.605	0.470 ± 0.094	4.320 ± 0.129	0.020 ± 0.005	16.230 ± 0.325
QER-LPD3QN	True	False	88.000 ± 1.320	1.310 ± 0.262	4.470 ± 0.067	0.010 ± 0.005	6.260 ± 0.125
IDDQN	True	False	97.000 ± 1.940	1.680 ± 0.336	4.700 ± 0.094	0.090 ± 0.005	8.930 ± 0.179

Values are mean ± standard deviation over repeated trials.

Table 7
Scenario: Right-turn narrow intersection – path planning results.

Algorithm	Success	Collides	Path length(m)	Min clearance(m)	Avg clearance	Path smoothness	Planning time (ms)
A*	True	False	86.790 ± 0.868	0.500 ± 0.100	2.220 ± 0.022	0.160 ± 0.005	18.880 ± 0.378
RRT*	True	False	91.160 ± 0.912	2.410 ± 0.482	5.410 ± 0.054	0.020 ± 0.005	86.920 ± 1.738
APF	True	False	87.370 ± 2.621	0.000 ± 0.050	2.300 ± 0.690	0.300 ± 0.005	3.240 ± 0.065
DWA-like	True	False	87.090 ± 2.613	0.440 ± 0.088	4.320 ± 0.129	0.020 ± 0.005	11.640 ± 0.233
QER-LPD3QN	True	False	90.000 ± 1.350	3.000 ± 0.600	3.950 ± 0.059	0.010 ± 0.005	8.770 ± 0.175
IDDQN	True	False	99.000 ± 1.980	1.020 ± 0.204	4.510 ± 0.090	0.080 ± 0.005	5.470 ± 0.109

Values are mean ± standard deviation over repeated trials.

seemingly competitive metrics, these are compromised by its significant failure rate.

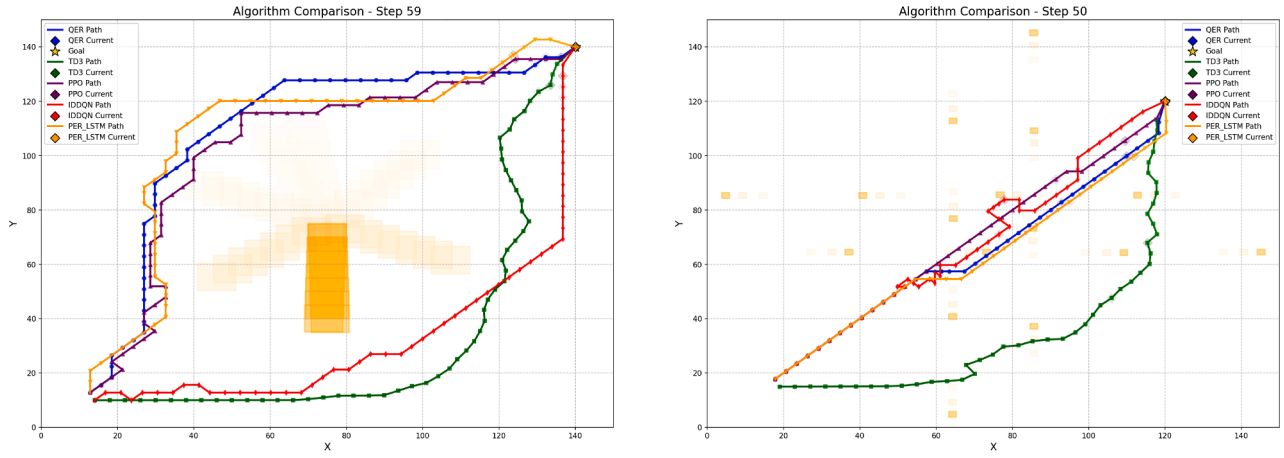
Among the successful planners, QER demonstrated the highest efficiency with the shortest path length of **220.56** and the best straightness ratio of **0.816**. This significantly outperforms the sequence-aware baseline PER_LSTM (Length: 233.19) and the memory-less TD3 (Length: 227.87), indicating that QER's strategy effectively avoids the "detour behavior" often seen in conservative planning. Regarding trajectory quality, QER achieved a smoothness score of 11.52, which is superior to IDDQN (14.52) and PER_LSTM (18.10), and remains highly competitive with the continuous-action baseline TD3 (10.71).

Real-time Feasibility Analysis: Table 8 also presents the computational cost. QER requires an average inference time of **3.68 ms**, which is higher than the lightweight TD3 (0.74 ms) due to the LSTM-based temporal feature extraction. However, this latency is notably lower than the stochastic PPO (6.00 ms) and corresponds to a control frequency of approx. **270 Hz**, well within the 10–50 Hz safety margin of autonomous systems. This confirms that QER successfully trades a negligible computational overhead for significant gains in path efficiency and success rates, making it suitable for deployment on vehicle computing units or Mobile Edge Computing (MEC) offloading scenarios.

5.1.2.2. Dynamic Traffic Flow Scene. A dense cross-traffic flow with multiple moving obstacles is illustrated in Fig. 9b. Multi-agent interaction and collision avoidance are required. Memory-aware methods achieve more stable passage selection in the narrow safe corridor. Although PPO can appear competitive in a snapshot, a non-negligible failure rate is observed over repeated trials. The quantitative results are reported in Table 9.

QER-LPD3QN maintained a **100% success rate** while achieving the optimal balance between efficiency and trajectory quality. It recorded a path length of **149.84** and a straightness ratio of **0.964**, outperforming the sequence-aware baseline PER_LSTM (Length: 151.81, Straightness: 0.952). While both methods leverage LSTM backbones for temporal feature extraction, QER-LPD3QN generates more direct paths, validating that the adaptive rotation operators in QER provide a more effective sampling distribution than classical TD-error prioritization. Furthermore, QER-LPD3QN achieved a smoothness score of **2.76**, representing a dramatic improvement over the memory-less baselines IDDQN (22.04) and TD3 (14.25), which struggled to infer obstacle intentions, resulting in oscillatory behaviors.

A critical analysis of the PPO baseline reveals the limitations of on-policy stochastic methods in this domain. Although PPO recorded the shortest path (146.70) and lowest smoothness value (1.83) in its successful trials, this performance is compromised by a **20% failure rate**. This suggests PPO adopts an overly aggressive strategy that trades safety for optimality, which is unacceptable for autonomous driving. Moreover, PPO exhibited the highest inference latency (**3.96 ms**), confirming that



(a) Scenario 1: Rotating-broom scene with periodic sweeping obstacles

(b) Scenario 2: cross-traffic flow scene with dense moving obstacles

Fig. 9. Trajectory visualization of different planners in two dynamic environments.

Table 8
Performance comparison of path planning algorithms in dynamic Rotating Broom scenes.

Algorithm	Success rate	Path length	Straightness	Smoothness	Reward	Infer. Time (ms)	Total Time (s)
QER	100%	**220.56 ± 3.62**	**0.816 ± 0.014**	11.52 ± 2.15	**11.73 ± 0.04**	3.68 ± 0.14	0.143 ± 0.006
TD3	100%	227.87 ± 0.01	0.794 ± 0.001	**10.71 ± 0.00**	11.61 ± 0.01	**0.74 ± 0.08**	**0.034 ± 0.003**
PPO	80%	250.14 ± 19.47	0.728 ± 0.035	34.99 ± 6.72	11.24 ± 0.20	6.00 ± 1.71	0.189 ± 0.033
IDDQN	100%	226.56 ± 2.48	0.799 ± 0.008	14.52 ± 2.07	11.64 ± 0.02	0.75 ± 0.06	0.033 ± 0.004
PER_LSTM	100%	233.19 ± 5.58	0.775 ± 0.018	18.10 ± 4.68	11.50 ± 0.10	1.14 ± 0.12	0.045 ± 0.004

Values are mean ± standard deviation. **Infer. Time**: Average inference time per step. **Total Time**: Total planning duration. Path smoothness: lower values indicate smoother paths. Straightness ratio closer to 1 indicates more direct paths. Success rate defined as percentage of trials reaching the goal. PPO excludes one failed trial for path metrics (n = 4 successful trials).

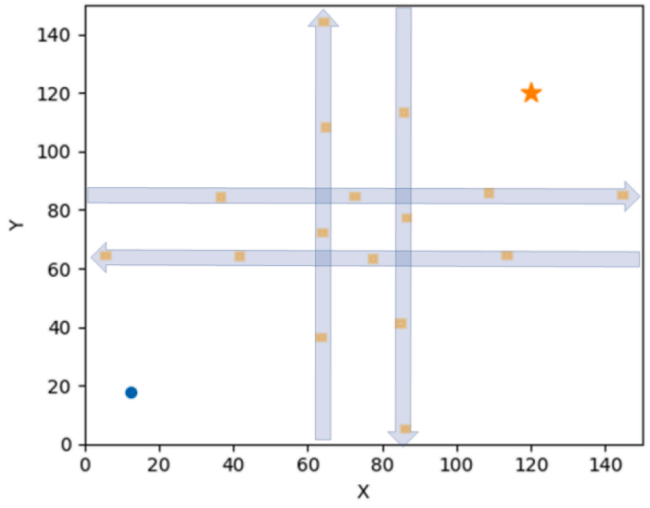


Fig. 10. Dynamic Interactive Intersection Diagram.

the computational overhead of stochastic sampling outweighs the benefits in real-time navigation.

Real-time Feasibility: As shown in Table 9, the proposed QER-LPD3QN demonstrates superior computational efficiency compared to PPO. Its average inference time of **3.15 ms** is approximately **20% faster** than PPO (3.96 ms). Although slightly slower than the simple MLP-based TD3 (0.79 ms), QER's latency supports a control frequency exceeding **300 Hz**, far surpassing the typical 10–50 Hz requirement for vehicle control. This confirms that QER-LPD3QN successfully integrates complex temporal reasoning without creating a computational bottleneck, making it highly suitable for deployment in time-sensitive edge computing environments.

5.1.2.3. Summary. The experimental results across both scenarios confirm that QER-LPD3QN achieves the optimal balance between safety, efficiency, and computational cost. While PER_LSTM performs adequately, QER-LPD3QN consistently generates shorter and straighter paths, outperforming both sequence-aware and memory-less baselines. Crucially, the real-time feasibility analysis demonstrates that the quantum-inspired "prepare" and "depreciate" operators incur only a marginal increase in latency (≈ 3 ms) compared to simple MLPs, yet support control frequencies exceeding 270 Hz. This validates that the proposed method effectively preserves temporal coherence without creating computational bottlenecks, making it a robust solution for real-time deployment in edge-enabled autonomous systems where PPO and other baselines falter.

5.1.2.4. Summary. The experimental results across both scenarios confirm that QER-LPD3QN achieves an optimal balance among safety, efficiency, and computational cost. While PER_LSTM performs adequately, QER-LPD3QN consistently generates shorter and smoother trajectories, achieving a 5.4% reduction in path length compared with the sequence-aware baseline under complex traffic conditions. Crucially, the real-time feasibility analysis shows that the quantum-inspired *prepare* and *depreciate* rotation operators introduce only a marginal latency increase (approximately 3 ms) compared with simple MLPs, while still supporting control frequencies exceeding 270 Hz. This demonstrates that the proposed method preserves temporal coherence without introducing computational bottlenecks, making it a robust solution for real-time deployment in edge-enabled autonomous systems where PPO and other baselines falter.

Table 9
Performance comparison of path planning algorithms in dynamic intersection scenes.

Algorithm	Success rate	Path length	Straightness	Smoothness	Reward	Infer. Time (ms)	Total Time (s)
QER	100.0%	149.84 ± 0.29	0.964 ± 0.004	2.76 ± 0.47	11.70 ± 0.01	3.15 ± 0.10	0.174 ± 0.005
TD3	100.0%	183.07 ± 0.01	0.796 ± 0.001	14.25 ± 0.01	11.30 ± 0.01	0.79 ± 0.05	0.045 ± 0.003
PPO	80.0%	146.70 ± 3.02	0.985 ± 0.020	1.83 ± 2.08	11.77 ± 0.04	3.96 ± 0.07	0.244 ± 0.008
IDDQN	100.0%	165.53 ± 2.35	0.873 ± 0.012	22.04 ± 2.62	11.43 ± 0.03	0.78 ± 0.03	0.044 ± 0.001
PER_LSTM	100.0%	151.81 ± 0.34	0.952 ± 0.002	2.67 ± 0.65	11.71 ± 0.01	1.34 ± 0.12	0.079 ± 0.007

Values are mean ± standard deviation. **Infer. Time**: Average inference time per step. **Total Time**: Total planning duration. Path smoothness: lower values indicate smoother paths. Straightness ratio closer to 1 indicates more direct paths. Success rate defined as percentage of trials reaching the goal. PPO excludes one failed trial for path metrics (n = 4 successful trials).

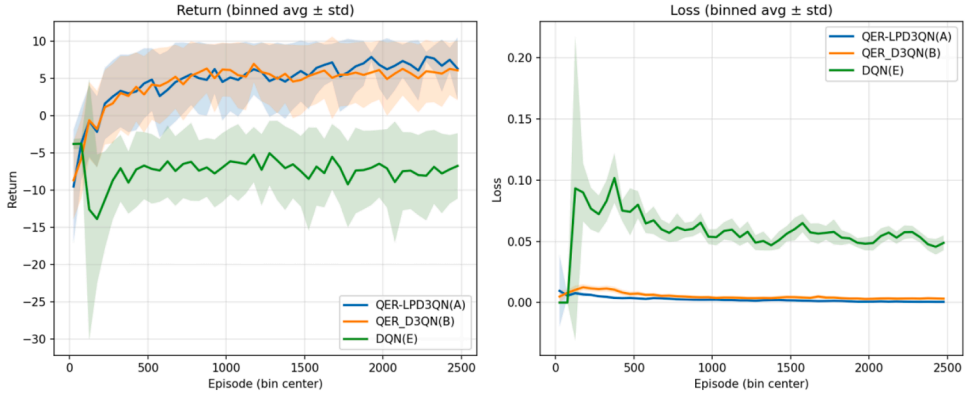


Fig. 11. A vs. B vs. E: QER-LPD3QN (A), QER-D3QN (B), and DQN (E).

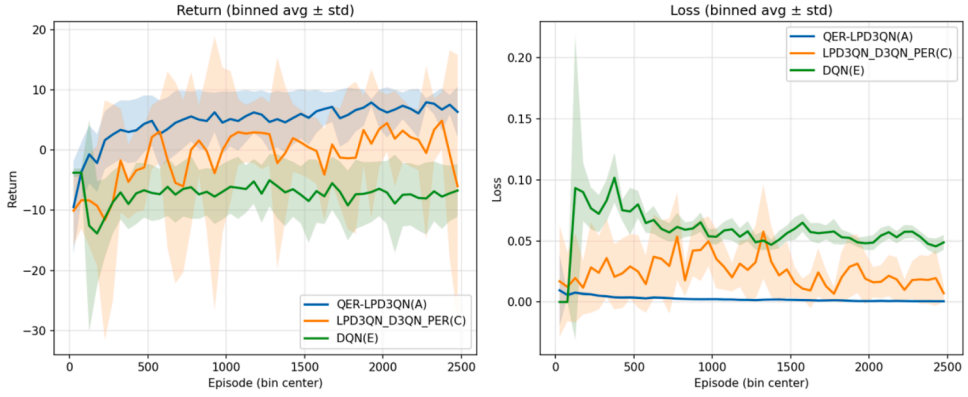


Fig. 12. A vs. C vs. E: QER-LPD3QN(A),LPD3QN-D3QN (PER)(C),and DQN(E).

5.2. Ablation study

To quantify the contribution of each module and validate their impact on learning efficiency and final performance, comprehensive ablation studies were conducted in a representative dynamic-interaction task, `cross_intersection`. The experiments were designed to systematically evaluate how different components influence both the final reward value and convergence behavior during training. For each ablation condition, only the target module was modified while keeping all other training and evaluation settings identical, ensuring fair comparisons across different configurations.

The scene is shown in Fig. 10. Four orthogonal traffic streams were placed at the center (two horizontal + two vertical). Reflective boundaries were used. Each dynamic agent moved at 5 m/s. The task stresses flow-crossing capability.

Five variants were compared: A (QER-LPD3QN), B (QER-D3QN), C (LPD3QN-D3QN with PER), D (D3QN with uniform replay, UR), E (DQN with UR). Each setting was trained with five random seeds. Mean ± standard deviation and 95% confidence intervals were reported.

Variant A achieved the highest or tied-highest success rate. A also yielded larger minimum clearances and smoother trajectories in dynamic scenes, with fewer samples required.

A vs. B (effect of LSTM). The comparison isolates temporal modeling under dynamic interference. A clear gain in obstacle-avoidance accuracy was observed when LSTM was enabled.

A vs. C (QER vs. PER). Relative to classical PER, QER improved sample efficiency and safety margins. Priority shaping at the sequence level provided more informative updates.

D vs. E (Dueling + Double). Dueling and Double reduced estimation bias and stabilized training, as reflected by lower loss and steadier returns.

Learning curves (Figs 11-13). The x-axis is the episode index. Shaded areas denote the across-seed standard-deviation bands.

Fig. 11 (Return/Loss). A and B rapidly moved from negative to positive return within 200-300 episodes, then entered a stable growth phase after 800-1000. Late-stage mean returns of A were slightly higher than B, with a narrower variance band, indicating higher convergence and better stability. E stayed negative with large oscillations and failed to

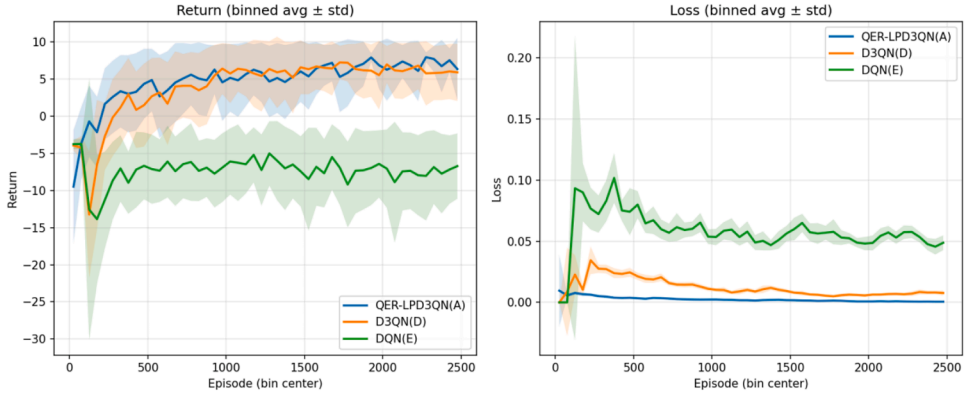


Fig. 13. A vs. D vs. E: QER-LPD3QN(A),D3QN(D),and DQN(E).

learn a viable crossing strategy. Loss curves showed that A/B reached near-zero loss very early and remained stable. E exhibited early spikes and converged slowly to a level still much higher than A/B, implying larger estimation error and learning noise. With QER fixed, adding LSTM (A vs. B) led to higher final return and improved stability. Pure DQN (E) struggled with strong nonstationarity.

Fig. 12 (Return/Loss). A dominated C from early training and maintained the advantage. C lagged in mean return and showed a wider, more oscillatory band. E again remained negative. Loss was lowest and most stable for A; higher with periodic ripples for C; and highest for E. Under matched backbones and schedules, QER yielded markedly better sample efficiency and stability than PER. The non-prioritized DQN baseline (E) was the weakest.

Fig. 13 (Return/Loss). Both A and D turned positive early and climbed. A reached a higher late plateau with smaller variance. E again stayed negative. Loss decreased fastest and to the lowest level for A; D followed and was clearly better than E; E remained high with early spikes. Dueling + Double (D) stabilized training and reduced bias. Adding QER + LSTM on top (A) further lowered loss and raised final return.

Across all three comparisons, A rose faster and converged higher. Against B (QER without LSTM), A showed a higher late plateau and tighter across-seed variance. Against C (PER), A maintained an all-phase advantage. The variance band of A shrank markedly in mid-to-late training, while C and E retained broader bands and were less stable. The loss of A decayed rapidly to near zero; C and D were higher and more oscillatory; E was highest with early spikes. Under strongly time-varying, pedestrian-crossing-like interactions, QER-LPD3QN (A) improved return level, convergence speed, and stability. The comparison with B verified the benefit of temporal modeling (LSTM). The comparison with C demonstrated the advantage of quantum-inspired replay (QER) over classical PER. The comparisons with D/E highlighted the necessity of Dueling + Double and the additive gains from QER + LSTM. The improvements in success rate, minimum clearance, and smoothness are consistent with the higher returns, lower losses, and reduced variance observed in the learning curves.

6. Conclusion and future work

A QER-LPD3QN planner was proposed for strongly dynamic environments. Sequence level, quantum inspired experience replay was coupled with an LSTM + Dueling + Double value architecture. Sampling probabilities of sequence samples were adaptively regulated via *prepare/depreciate* rotations. Temporal coherence, sample efficiency, and estimation stability were jointly preserved.

Comprehensive comparisons were conducted in dense obstacle maps, narrow intersections, and two highly dynamic scenarios. Stable advantages were observed over A*/RRT*/APF/DWA-like and DQN family baselines in success rate, safety clearance, and trajectory smoothness, while maintaining millisecond level online planning. Ablations indicated that sequence level QER contributed the primary gain in sample efficiency, and LSTM provided the main benefit for long horizon temporal modeling. With minor adaptations, the method can be deployed on UAVs, autonomous vehicles, and other mobile agents.

The present study was limited to simulation due to time and resource constraints. Future work will migrate the algorithm to real platforms, validate with onboard sensors, and investigate real time deployment of reinforcement learning under real world dynamics.

CRediT authorship contribution statement

He Guo: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft, Visualization; **Zhengyi Chai:** Supervision, Validation, Writing – review & editing; **Yalun Li:** Project administration, Resources, Funding acquisition.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work was supported by the National Natural Science Foundation of China (Grant no. 62172298). Fujian Province Young and Middle aged Teacher Education Research Project (Science and Technology Category)(No. JZ240101).Open Project of Tianjin Key Laboratory of Tightening Connections (TKLFJ2025-02-B-03).

References

Ansere, J. A., Gyamfi, E., Sharma, V., & et al. (2023). Quantum deep reinforcement learning for dynamic resource allocation in mobile edge computing-based IoT systems. *IEEE Transactions on Wireless Communications*, *23*(6), 6221–6233.

Bai, Z., Pang, H., He, Z., & et al. (2024). Path planning of autonomous mobile robot in comprehensive unknown environment using deep reinforcement learning. *IEEE Internet of Things Journal*, *11*(12), 22153–22166.

Carvalho, J., Le, A. T., Kicki, P., & et al. (2025). Motion planning diffusion: Learning and adapting robot motion planning with diffusion models. *IEEE Transactions on Robotics*.

Dai, J. Y., & Zhou, N. R. (2023). Optimal quantum image encryption algorithm with the QPSO-BP neural network-based pseudo random number generator. *Quantum Information Processing*, *22*(8), 318.

Deng, W., Cai, X., Wu, D., & et al. (2024). Moqea/d: Multi-objective QEA with decomposition mechanism and excellent global search and its application. *IEEE Transactions on Intelligent Transportation Systems*, *25*(9), 12517–12527.

Fan, J., Chen, X., & Liang, X. (2023). UAV Trajectory planning based on bi-directional APF-RRT* algorithm with goal-biased. *Expert Systems with Applications*, *213*, 119137.
Gök, M. (2024). Dynamic path planning via dueling double deep q-network (D3QN) with prioritized experience replay. *Applied Soft Computing*, *158*, 111503.
Hakemi, S., Houshmand, M., KheirKhah, E., & et al.(2024). A review of recent advances in quantum-inspired metaheuristics. *Evolutionary Intelligence*, *17*(2), 627–642.
Huang, C., Guo, Y., Zhu, Z., & et al. (2024). Quantum exploration-based reinforcement learning for efficient robot path planning in sparse-reward environment. In *Proceedings of the 33rd IEEE international conference on robot and human interactive communication (RO-MAN)* (pp. 516–521).
Li, H., Zhong, P., Liu, L., & et al. (2025). Robot dynamic path planning based on prioritized experience replay and LSTM network. *IEEE Access*.
Li, Y., Aghvami, A. H., & Dong, D. (2021). Intelligent trajectory planning in UAV-mounted wireless networks: A quantum-inspired reinforcement learning perspective. *IEEE Wireless Communications Letters*, *10*(9), 1994–1998.
Olvera, C., Montiel, O., & Rubio, Y. (2023). Quantum-inspired evolutionary algorithms on continuous space multiobjective problems. *Soft Computing*, *27*(18), 13143–13164.
Tang, G., Tang, C., Claramunt, C., & et al. (2021). Geometric a algorithm: An improved a algorithm for AGV path planning in a port environment. *IEEE Access*, *9*, 59196–59210.
Wang, D., Song, B., Lin, P., & et al. (2021). Resource management for edge intelligence (EI)-assisted IoV using quantum-inspired reinforcement learning. *IEEE Internet of Things Journal*, *9*(14), 12588–12600.
Wang, L., Zhang, G., Yang, Q., & et al. (2025). An adaptive traffic signal control scheme with proximal policy optimization based on deep reinforcement learning for a single intersection. *Engineering Applications of Artificial Intelligence*, *149*, 110440.
Wei, Q., Ma, H., Chen, C., & et al. (2021). Deep reinforcement learning with quantum-inspired experience replay. *IEEE Transactions on Cybernetics*, *52*(9), 9326–9338.
Wei, X., Gao, X., Ye, K., & et al. (2024). A quantum reinforcement learning approach for joint resource allocation and task offloading in mobile edge computing. *IEEE Transactions on Mobile Computing*.
Yu, H., Zhao, X., & Chen, C. (2024). Quantum-inspired reinforcement learning for quantum control. *IEEE Transactions on Control Systems Technology*.
Zhang, D., Shi, W., & St-Hilaire, M. (2025a). The permissioned blockchain-based quantum-inspired edge intelligence approach for the services of future internet of vehicles. *IEEE Transactions on Intelligent Transportation Systems*.
Zhang, R., Guo, H., Andriukaitis, D., & et al. (2024). Intelligent path planning by an improved RRT algorithm with dual grid map. *Alexandria Engineering Journal*, *88*, 91–104.
Zhang, R., Guo, H., Sotelo, M. A., & et al. (2025b). New RRT-based method for vehicle path planning in curve scenarios considering path oscillations. *IEEE Transactions on Vehicular Technology*.
Zhou, Y., Yang, J., Guo, Z., & et al. (2024). An indoor blind area-oriented autonomous robotic path planning approach using deep reinforcement learning. *Expert Systems with Applications*, *254*, 124277.